INTERNAL & AUDITOR REPORT

fUSD Core Upgrade: Verification & Rehearsal

Migration of the live Base-mainnet fUSD deployment from the `audit` baseline to `feature/09-softstack-stage2-fixes` — upgrade methodology, every new contract involved, and exhaustive functional test evidence gathered against a real, forked copy of live mainnet state.

Prepared for: Internal engineering team & SoftStack auditors **Network:** Base mainnet (chain 8453)

Investment strategy: Stablecoin deposits into lending markets only — no borrowing, no volatile assets

---

Baseline commit: `f6882b9` (`audit`) **Target commit:** `ceff2c7` (`feature/09-softstack-stage2-fixes`)

CONTENTS

01 Executive summary
02 Scope & objective
03 Upgrade methodology
04 Live custody re-verification
05 New deployed contracts
06 Storage-layout verification
07 Core user flows
08 Lending guard coverage (Morpho Blue, Aave V3)
09 Exhaustive withdrawal-function coverage
10 Attested withdrawal plan
11 Guard validation (negative tests)
12 Multi-user isolation & fairness
13 Automatic pro-rata withdrawal (asset guards)
14 ERC20Guard validation
15 New protections introduced by this upgrade
16 Access control (unauthorized-caller rejection)
17 Reinitializer protection & upgrade-script idempotency
18 Stablecoin-only Uniswap V3 testing & the artificial Aave debt scenario
19 A second independent finding: NftTrackerStorage misconfiguration
20 Comprehensive roles & access-control map
21 Role-scoped testing (user / trader / manager) & migration-safety verification
22 Manager fee-minting & fee-governance testing
23 Self-review: fork fidelity & testing-completeness verdict
24 Systematic guard-by-guard audit, plus the three dormant guard families proven against real infrastructure
25 Issues found and resolved during rehearsal
26 External findings
27 Outstanding items before real execution
28 Reproducing this rehearsal

01 Executive summary

The upgrade from the live `audit`-branch deployment to `feature/09-softstack-stage2-fixes` — carrying every fix from the SoftStack Stage 1 and Stage 2 audit rounds — was rehearsed in full against a local fork of real, live Base mainnet state, not a synthetic test fixture. Both halves of the upgrade (the four core proxy contracts, upgraded in place, and six newly-deployed, non-upgradeable guard/manager contracts the audit report flagged as code-merged-but-not-yet-live — closing five distinct findings between them) were executed end to end, signed by an impersonated copy of the real on-chain DAO Safe, with zero private keys and zero mainnet transactions sent. A fifth proxy upgrade — `NftTrackerStorage`, fixing an independent, already-CertiK-flagged bug this rehearsal found live on real mainnet (Section 19) — was added to the same batch mid-engagement.

Two further rounds then covered the full protocol surface the fund actually uses: every withdrawal entry point on `PoolLogic` (including the attested/selective withdrawal plan path), the two active lending integrations (Morpho Blue and Aave V3, supply/withdraw only — this fund does not borrow or hold volatile assets), guard-layer validation for the Uniswap-related fixes that exist in the codebase but are not part of the fund's own strategy, multi-user isolation and fairness with three independent accounts exiting through three different withdrawal paths simultaneously, the asset guards' own automatic pro-rata withdrawal logic against real, persistent Morpho and Aave positions, and `ERC20Guard`'s dispatch-layer validation. A further round confirmed every mandatory post-upgrade initializer is genuinely non-replayable, found and fixed a real operational gap in one of the upgrade scripts' own re-run safety, and confirmed zero regressions across the repository's full pre-existing test suite. A precision pass re-verified exactly which of the six new guard contracts were actually exercised on the fork and by what mechanism — catching and correcting a wrong attribution (Section 5) and disclosing one guard (`UniswapV3NonfungiblePositionGuard`, M-14) with zero fork coverage at the time. A final round then unblocked exactly that gap: a real, live USDC/USDT_BASE stablecoin pair enabled a genuine Uniswap V3 swap and the full LP-minting lifecycle (M-14 included), plus a deliberately artificial Aave debt scenario for M-08's one remaining untested branch — and surfaced two independent, real findings along the way: a second, previously-live, CertiK-flagged `NftTrackerStorage` misconfiguration (now fixed, Section 19), and the fact that every stablecoin on the real Aave V3 Base deployment currently carries 0% LTV (Section 18). A comprehensive, live roles-and-access-control map across every touched contract closes out this round (Section 20).

BOTTOM LINE

71 of 72 tests pass. The one failure is a real, deliberately-sought finding (the pre-existing `commitFeeIncrease()` gap below), not a flaw in the rehearsal. The repository's full existing suite — 1,366 tests — also passes clean against the target branch, zero new regressions. The upgrade, as scripted, is sound and ready for a final team/auditor review of the **three** Safe transaction batches it now produces across two separate Safes. Nothing in this rehearsal was broadcast to real mainnet.

What was confirmed, by round:

- **Migration safety.** Total fund value, fUSD supply, and one real staking position's individual pending reward all preserved exactly (to the decimal) across the actual upgrade transition.
- **Every withdrawal path** — including nonce-replay and expired-signature rejection on the attested plan path — and three independent users exiting simultaneously through three different paths with zero cross-contamination.
- **New protections have real teeth** — the deposit cap, the EUR-mode lock, the sequencer uptime feed — and every privileged function correctly rejects an unauthorized caller, including the trader role. Every mandatory initializer is non-replayable (Section 17).
- **Finding 1 — fixed:** a second, independent `NftTrackerStorage` misconfiguration already flagged by CertiK, found live and fixed (Section 19), alongside a real USDC/USDT_BASE Uniswap V3 swap + LP-minting lifecycle proof (Section 18) and a full live roles-and-access map (Section 20).
- **Finding 2 — disclosed, not fixed here:** `commitFeeIncrease()` silently zeroes every fee when nothing is announced, instead of reverting. Confirmed identical on both branches — unrelated to this upgrade, not a fund-safety issue, but a real manager-revenue footgun (Section 22).
- **Finding 3 — fixed:** a systematic, guard-by-guard pass (Section 24) found `UniswapV3AssetGuard` carrying three closed fixes (M-06, FNA-16, FNA-58) that neither upgrade script had redeployed. Fixed and covered by six new tests, including a follow-up that confirmed M-06's own check is independently reachable, not shadowed dead code.
- **Finding 4 — disclosed, not fixed here:** `MerklRewardClaimGuard` (FNA-19) is also live and un-redeployed, left as-is because its real on-chain registration target could not be conclusively confirmed.
- **Three previously-dormant guard families, proven functionally correct** against real, live external infrastructure (Aave V4's actual Base deployment, a genuine Morpho Vault V2 via the real factory) — 15 new tests, not mocks.
- **Finding 5 — a methodology gap in this rehearsal's own tooling, found and fixed:** that same dormant-guard round initially mispriced one of the three families by ~10^{12}× from testing against the pre-upgrade pool, which doesn't understand the pre-valued-guard system these guards need. Traced to its root cause and fixed by testing against the upgraded pool instead; all three scripts now refuse to run against the wrong baseline again.

02 Scope & objective

The live fUSD pool on Base has been running the pre-audit `audit` branch implementation since its original deployment, holding real, user-deposited funds. Since then, two independent audit rounds (CertiK, then SoftStack Stage 1 and Stage 2) produced a long list of fixes, all merged to `feature/09-softstack-stage2-fixes` but never deployed to the live contracts. The objective of this work was to:

1. Design and build the actual upgrade transactions needed to bring the live deployment to that branch.
2. Prove, against a real copy of live state, that those transactions execute cleanly and preserve every user's funds and position exactly.
3. Exhaustively exercise every user-facing and guard-level function the upgraded contracts expose, scoped correctly to how the fund actually operates, to confirm nothing regressed and every audit fix is genuinely live.
4. Produce this report as the evidence record for internal sign-off and for the auditors who reviewed the code being deployed.

> **STRATEGY SCOPE, STATED EXPLICITLY**
>
> This fund's investment strategy is stablecoin deposits into lending markets only — it does not borrow, and it does not hold or trade volatile assets. Testing in this report is scoped accordingly: Morpho Blue and Aave V3 are tested with real supply/withdraw activity (the fund's actual exposure); Uniswap V3 swap and liquidity-provision guards are tested only at the validation layer (confirming they correctly reject bad input), since minting a real position or executing a real swap would require a volatile asset this fund does not and will not hold, and fabricating one would misrepresent the fund's real strategy in a report auditors will rely on.

This report does not cover the planned Euro-denominated product, which is a fresh deployment rather than an upgrade and is tracked separately.

03 Upgrade methodology

Why a local fork, and why Anvil specifically

Rehearsing a real upgrade against a byte-for-byte copy of live chain state — including the real DAO Safe, the real token balances, the real staker positions — is the only way to find a migration bug before it reaches real funds. Hardhat's own built-in forking was tried first and found to have a genuine, reproducible limitation on Base (an OP-stack L2): any contract call against a Base fork fails with "No known hardfork for execution on historical block … in chain with id 8453", regardless of configuration. Foundry's **Anvil** forks Base correctly — contract execution, account impersonation and transaction broadcasting all work exactly as they would against the real network, just locally. The rest of this rehearsal uses Anvil exclusively, pointed at via a new `baseFork` Hardhat network entry that only ever talks to `127.0.0.1`.

Two-script, two-phase design

SCRIPT	COVERS
`upgrade_softstack_core_contracts.ts`	The four upgradeable proxies (AssetHandler, PoolManagerLogic, PoolLogic, TokenLogic) and their mandatory post-upgrade initializers.
`upgrade_softstack_guards.ts`	The five guard/manager contracts flagged by the audit as merged-but-not-live, plus the custody cleanup described in Section 4.

Each script runs in two phases. **Phase 1** (permissionless) deploys every new contract and runs read-only safety checks. **Phase 2** (owner-gated) is never sent automatically: by default each script writes a Gnosis Safe Transaction Builder JSON file to disk, meant to be reviewed and proposed through the Safe UI. A script only broadcasts directly when explicitly run with `SEND=1`, and it refuses to do that against real mainnet at all unless a second, separate override is also set — this rehearsal never set that override.

04 Live custody re-verification

Every ownership and admin role relevant to this upgrade was read directly from the live chain before anything was built, not trusted from prior notes.

> **CUSTODY HAS CONSOLIDATED**
>
> All five privileged roles across the four core proxies and the Governance registry are now held by a single address: the genuine 3-of-4 Gnosis Safe at `0x74aF72D91D5FB263fBa09Ed43aD1C1ea079058B3`. Confirmed live on-chain. This simplifies the upgrade to a single Safe proposal.

GAP FOUND, ADDRESSED BY THIS UPGRADE

`MorphoBlueManager` and `SlippageAccumulator` are still owned by the pool manager Safe (`0x18f84Ea9…`) rather than the DAO Safe — exactly the configuration the audit's M-11 finding flagged as a live risk. A third contract, `AaveV3LendingPoolAssetGuard` , was found in the same state despite not being named in that finding. The guard-redeployment script transfers ownership of all three to the DAO Safe as part of the same batch.

05 New deployed contracts

Addresses below are from this rehearsal's fork run, for evidence purposes — the real mainnet deployment will mint different addresses, but every contract listed is the exact, already-compiled `feature/09-softstack-stage2-fixes` bytecode that will be deployed for real.

Core contracts batch

All four contracts below are genuine proxy upgrades: `AssetHandler` , `PoolManagerLogic` , `PoolLogic` , and `TokenLogic` each keep their existing, live proxy address (listed in Section 4/6) — only the implementation each proxy points to changes, via each proxy's own `ProxyAdmin` (or, for the UUPS `TokenLogic` , its own `upgradeToAndCall`). `FrgmntUserActions` and the four libraries are the only genuinely new deployments in this batch, needed to support the upgraded implementations.

CONTRACT	ROLE	FORK ADDRESS
AssetHandler	New implementation	`0xE8493406aDc19A02f1c3AA722623deAcd7DF6eF8`
PoolManagerLogic	New implementation	`0xf1485e28070983B2145D262F110Ae1b62f9FBC6b`
TokenLogic	New implementation	`0x42E4Ffc8b728D42E4A2CA3d87c8881bd007E5AbE`
PoolLogic	New implementation	`0xA0401A31226a90F6379baEfD47C405aC52C206B7`
FundCalculationLibrary	Redeployed library	`0x7Be6E341624D7f730f91CAa7867416D63DbEA219`
CallResultChecker	Redeployed library	`0x9eaEb6Ffb89E80A5f7e926ef7cee0831C9348516`
PoolTxExecutor	Redeployed library	`0x5c7e41E0bDc3C406a3c99BC5f03D79e8cABE7Cad`
WithdrawalPlanLib	New library	`0x9EEaBc0705aecfd61dDb488598261EB11e7C2438`
WithdrawalEscrow	New, bound to the pool proxy	`0x9Ba9F602B84D46f9c511518520aFb773162A0df3`
FrgmntUserActions	New — sole intended holder of `userActionsRouter` (closes M-13)	`0xE55d12Bb458F0dcF4E3Ed9703943F972812c2c76`

Guard contracts batch

None of the six contracts below are upgradeable proxies — unlike the four core contracts above, there is no "upgrade in place." Each is a fresh, standalone deployment at a brand-new address; "closing" a finding here means redeploying the fixed contract and then re-pointing `Governance` 's registry entry for that slot (`contractGuards(target)` or `assetGuards(assetType)`) at the new address. The old instance is simply abandoned, not migrated or destroyed.

CONTRACT	FINDING CLOSED	FORK ADDRESS	EXERCISED ON THIS FORK?
SlippageAccumulator	Fixes the pool-factory wiring bug blocking every router swap since deployment	`0x44405E7D8C89881dAC6f8dfEe13d3C7562EA4505`	**Partial** only the upgrade script's own post-deploy sanity probe (`assetValue()` , no revert) — see Test 15
UniswapV3RouterGuard	M-09 / M-12 — per-swap output floor, multi-hop validation	`0x17761f8b0640ec4A2a38C9FBFe979Dbbf91ed795`	**Yes** — real `execTransaction` dispatch, Tests 16 & 17 (rejection cases only — see Section 2's strategy-scope note for why no real swap)
UniswapV3NonfungiblePositionGuard	M-14 — LP mint/increaseLiquidity loss accounting	`0x28f693F3AdE67Cab21123D42e6FF39889588Dc3D`	**Yes** — full real mint/increaseLiquidity/decreaseLiquidity/collect/burn lifecycle, Section 18, Tests 34–38 (originally disclosed as zero coverage; see the update callout below)
MorphoBlueManager	M-11 — rejects a pool whose manager/trader is the contract's own owner	`0x495506Ac55bCD67F95D8AA6F8A78a6Ae5E63C566`	**Yes** — Test 18 (config readback) and Test 6 (real supply/withdraw dispatch)
MorphoBlueContractGuard	Bound to the new manager above	`0x23c04434e7847640e5c3982C2292677C11541A5A`	**Yes** — Test 6's real `execTransaction` dispatch for an actual supply + withdraw
AaveV3LendingPoolAssetGuard	M-08 — degrades NAV instead of reverting the whole pool on an unconfigured swap route	`0xc396EF69582127D55E974C59CFf2d0fa364494ce`	**Yes** — Test 22's direct `getBalance()` call plus the pro-rata withdrawal's internal `withdrawProcessing()` call. **Not** Test 7 — see the correction below.

The new `MorphoBlueManager` instance was not deployed empty: the live pool's real, existing Morpho market configuration — **10 market IDs** — was read from the current instance and replayed onto the new one before registration, confirmed matching exactly.

CORRECTION — TEST 7 (AAVE SUPPLY + WITHDRAW) DOES NOT EXERCISE THE NEW AAVEV3LENDINGPOOLASSETGUARD

Section 8's original M-08 callout stated "Test 7 above exercises the real, reachable surface" of this guard. That was imprecise to the point of being wrong, caught on direct re-review: Test 7's `execTransaction(AAVE_POOL, ...)` calls are dispatched through whichever **contract guard** is registered for the Aave V3 Pool address — `AaveLendingPoolGuardV3` (`0x7Ef5442f796bF1Ae3e00E91a5527cAa5F7aba5A4` , confirmed live via `Governance.contractGuards()`) — a completely separate contract from `AaveV3LendingPoolAssetGuard` , **not touched by this upgrade at all** (it is not in the guards script's deploy list, and its registration is untouched by the Phase 2 batch). The M-08 fix lives in the **asset** guard (`Governance.assetGuards(2)`), which governs NAV valuation and pro-rata withdrawal (`getBalance` / `withdrawProcessing`) — a genuinely different code path from the contract-guard dispatch Test 7 exercises. The real evidence for the new asset guard is Test 22 (Section 13), not Test 7. The M-08 callout immediately below has been corrected to reflect this.

TWO GUARDS THIS UPGRADE DOES NOT TOUCH, BUT THAT THE FORK TESTS INCIDENTALLY EXERCISE — DON'T MISTAKE THESE FOR NEW-CODE EVIDENCE

`AaveLendingPoolGuardV3` (above) and `MorphoBlueAssetGuard` (`Governance.assetGuards(1)` , `0x27BeceFb6CF59b26CD73dac227Ae3597065E2850` , confirmed live) are both pre-existing, unchanged contracts — neither is part of this upgrade's deploy or registration list, confirmed by reading the guards script directly and by reading each address back from a live `Governance` call on a fresh, untouched fork. Test 7's Aave dispatch runs through the first; Test 22's Morpho `getBalance()` call runs through the second. Both tests still correctly prove the pro-rata/dispatch mechanism works end to end against real, live state — just not as evidence for any new SoftStack fix in those two specific contracts, which this upgrade doesn't change.

UPDATE — M-14 NOW HAS REAL FORK COVERAGE (SUPERSEDING THE GAP ORIGINALLY DISCLOSED HERE)

This section originally disclosed that no test exercised `UniswapV3NonfungiblePositionGuard` at all, since minting against a real Uniswap V3 pool appeared to require a volatile asset outside this fund's strategy. That constraint turned out to be avoidable with a real stablecoin pair (USDC/USDT_BASE) — see Section 18 for the full real mint/increaseLiquidity/decreaseLiquidity/collect/burn lifecycle, now covered end to end (Tests 34–38, all passing). The dedicated unit suite cited below remains additional, independent evidence, not a replacement.

DEDICATED UNIT-TEST SUITE FOR THIS GUARD — STILL CITED, STILL PASSING

`npx hardhat test test/UniswapV3NonfungiblePositionGuard.test.ts` — **18 of 18 tests pass** (re-run directly to confirm, not cited from memory).

A fifth proxy upgrade, found mid-engagement: NftTrackerStorage

Unlike the six guards above, this is a genuine proxy upgrade — same pattern as the four core contracts in Section 5's first table, not a fresh standalone deployment. Added to the DAO Safe's guard batch after Section 19's finding; see that section for the full writeup of why.

CONTRACT	FINDING CLOSED	FORK ADDRESS (IMPLEMENTATION)	EXERCISED ON THIS FORK?
NftTrackerStorage	Independent, CertiK-flagged `poolFactory` misconfiguration — see Section 19	`0x495506Ac55bCD67F95D8AA6F8A78a6Ae5E63C566`	**Yes** — every one of Tests 34–38 depends on this fix; confirmed by re-running the full suite both before (failing) and after (passing) it landed

06 Storage-layout verification

Every one of the four core proxies had its storage layout diffed field-by-field between the `audit` baseline and `feature/09-softstack-stage2-fixes` , using `forge inspect storage-layout` against both branches checked out in separate worktrees — a tool-verified slot/offset comparison, not a visual code read.

CONTRACT	RESULT	NEW FIELDS
AssetHandler	PASS	`eurUsdAggregator` , `eurUsdTimeout` , `eurUsdModeLocked` — 3 slots of its reserved gap
PoolManagerLogic	PASS	`userActionsRouter` — 1 slot of its reserved gap
TokenLogic	PASS	`maxDepositFusdSupply` , `protocolFusdOutstanding` — 2 slots of its reserved gap
PoolLogic	PASS	18 new variables, strictly appended after every pre-existing one (no reserved gap by design — append-only ordering holds)

07 Core user flows

Run against the fully upgraded fork, as a brand-new test account with no pre-existing position — the strictest test, since an account that already had state before the upgrade could mask a migration bug behind data that was already correct.

#	TEST	RESULT	EVIDENCE
1	Deposit	PASS	Deposited 10,000 USDC, received 9,997.3668 fUSD
2	Stake	PASS	Staked 9,997.3668 fUSD, received 9,997.3668 sfUSD
3	Harvest (auto-compounding migration path)	PASS	`compoundedRewardIndex` moved from `1.000000262…e18` to `1.000058761…e18` after 6 hours of real accrued Morpho interest — the specific migration math the original (pre-rewrite) upgrade script's own notes had flagged as never independently verified
4	Unstake	PASS	Unstaked all 9,997.3668 sfUSD, received the matching fUSD back
5	Immediate withdrawal	PASS	Withdrew 4,998.98 fUSD, received 5,000.29 USDC back, after correctly clearing the 1-day exit cooldown

> **FUND INTEGRITY, CONFIRMED DIRECTLY**
>
> Total fund value and total fUSD supply were read immediately before and immediately after the core-contracts upgrade, before any test activity began: both were preserved exactly, to the wei.

08 Lending guard coverage

The fund's actual, live exposure: real supply and withdraw activity against both active lending integrations, using the pool's real configuration — not a synthetic market.

#	TEST	RESULT	EVIDENCE
6	Morpho Blue supply + withdraw	PASS	Used market `0x9103c3b4…`, a real, governance-approved market — one of the pool's 10 live markets, confirmed to be a USDC-denominated loan market. Supplied 1,000 USDC (gained 899,261,812,931,645 supply shares), withdrew the full position back, `supplyShares` returned to exact pre-test baseline.
7	Aave V3 supply + withdraw (lending only, no borrow)	PASS	Supplied 1,000 USDC (test's own delta: aToken balance +1,000.000248); the subsequent `withdraw(asset, type(uint256).max, pool)` correctly withdrew the pool's **entire** Aave aToken balance (per Aave's own semantics for a max-amount withdraw, not just this test's own 1,000) — see the correction callout immediately below for why that balance was not 1,000.

All 10 of the pool's real Morpho markets share the same `loanToken` : USDC — confirmed by reading each market's full `MarketParams` directly, not assumed. No `borrow` / `repay` / `supplyCollateral` calls were made against either protocol, consistent with the fund's lending-only strategy.

CORRECTION, RE-VERIFIED 2026-10-31 — THE LIVE POOL'S AAVE V3 POSITION IS NOT EMPTY

An earlier round of this report (and a header comment in `upgrade_softstack_guards.ts`) stated the live pool's Aave V3 position was "currently empty," inferred from `AaveV3LendingPoolAssetGuard.uniV3Fee(WETH, USDC)` reading 0 (unconfigured). That inference was wrong: reading the pool's actual aToken balance directly from `AaveProtocolDataProvider` — not inferred — shows the live pool currently holds a real, substantial USDC supply position in Aave V3: **~$109,129**, against a total fund value of ~$121,266 and a Morpho Blue supply of ~$12,166 — i.e. the large majority of this fund's current TVL is deployed in Aave V3 right now, not idle or in Morpho. This is a genuine, live change on real mainnet since this report's figures were first captured, not a test artifact (re-confirmed on a completely untouched fresh fork, before running any test activity). It does not change this upgrade's safety: Aave variable debt for USDC is confirmed 0 (no borrowing, consistent with this fund's lending-only strategy), and the swap-route configuration M-08 cares about is only relevant to settling Aave V3 debt, not a supply position — so the "dormant under this fund's strategy" conclusion below still holds, for the reason actually stated there (debt-gated, not position-size-gated). The earlier, less precise phrasing has been corrected in both this report and the script's own header comment.

M-08'S SPECIFIC FIX — DORMANT UNDER THIS FUND'S STRATEGY, CONFIRMED BY CODE

M-08 (`AaveV3LendingPoolAssetGuard`) fixes NAV computation freezing when an Aave debt asset is listed without a configured settlement swap route. Reading `_netRealizableBalance` directly: the entire route-check this fix touches is gated behind `debtAssetCount == 0 → return gross` — it is physically unreachable unless the pool holds real Aave V3 debt. Since this fund's strategy excludes borrowing, that code path cannot execute in production, now or after this upgrade, regardless of the Aave supply position's size (see the correction above). The real, reachable surface this guard DOES expose under the current strategy — `getBalance` / `withdrawProcessing` against a large, real supply position — is exercised correctly by Test 22 (Section 13), not Test 7 (see the correction above: Test 7 dispatches through an entirely different, untouched contract guard). The debt-gated degrade-vs-revert behavior itself was not re-derived against a live debt position in this round, for the same reason the Uniswap LP/swap guards were not — it does not reflect how this fund operates. The fix remains correct and already-merged code, confirmed present by the storage/bytecode check in Section 5; it simply has no live trigger condition under the current strategy.

09 Exhaustive withdrawal-function coverage

Every withdrawal entry point exposed by `PoolLogic`, tested individually and in full, as the user explicitly requested.

#	FUNCTION	RESULT	EVIDENCE
8	`withdrawCashImmediateTo` (third-party recipient)	PASS	Caller burned 2,999.26404 fUSD; the named third-party recipient received 2,999.999998 USDC; the caller's own USDC balance was untouched.
9	`withdrawCashImmediateSafe` (explicit `ComplexAsset[]`)	PASS	Called with a 4-entry array matching `getSupportedAssets().length`; withdrew 2,999.26404 fUSD for 2,999.999998 USDC.
10	`withdrawCashImmediateToSafe` (recipient + complex data combined)	PASS	Remaining 5,998.52808 fUSD withdrawn to a third party; caller's fUSD balance left at exactly 0.
11	Queued withdrawal — full 3-step chain: `requestCashWithdraw` → `finalizeCashWithdraw` → `claimCashWithdraw`	PASS	Request #1 created, finalized by the impersonated pool manager, then claimed: requested 4,998.7734 fUSD worth, claimed 5,000.0 USDC. Run on its own isolated snapshot (immediate withdrawal must be disabled for the queue to accept requests) so it does not interfere with the immediate-withdrawal tests above.

10 Attested withdrawal plan

The most involved withdrawal path: an off-chain attester signs a specific, bounded withdrawal composition (EIP-712), which the user then submits on-chain. Neither upgrade script configures an attester by default, so this was set up from a clean slate: a fresh, test-only keypair was generated and registered as the pool's attester (impersonated DAO Safe call), with a real, non-zero volume cap so a genuine plan could execute — nothing here touches a real key or real mainnet state.

#	TEST	RESULT	EVIDENCE
12	Happy path	PASS	Plan nonce 1, signed by test attester `0x7c8dfD58…` : 100.0 fUSD burned, 100.024537 USDC received.
13	Replay rejection	PASS	Resubmitting the identical already-used signed plan reverts `PlanNonceAlreadyUsed()` (selector `0x599bd9b1` , confirmed matched against the raw revert data) — genuine on-chain replay protection, not a coincidental earlier-stage failure.
14	Expired-deadline rejection	PASS	A plan with `deadline` set 1 second in the past reverts `PlanDeadlineExpired()` (selector `0x4b8f2ff6` , confirmed matched).

The reference attester service's own plan-composition and EIP-712 signing logic (`services/attester/src/composer.ts` , `signer.ts`) was reused rather than hand-rolled, reducing the risk of a subtle encoding bug producing a false pass.

> **COVERAGE GAP, DISCLOSED DIRECTLY — ONLY THE BASIC REPLAY/EXPIRY SAFETY PROPERTIES ARE TESTED, NOT THE PLAN MECHANISM'S RICHER BRANCHES**
>
> The three tests above exercise exactly one plan shape: a single, whole-asset, portion-based allocation, well within every bound, with negligible volume pressure. Reading `WithdrawalPlanLib.executeWithdrawalPlan()` 's own source directly (done later, in Section 24's investigation) surfaces real branches this report does not exercise at all:
>
> - **Multiple allocations in one plan**, across different assets.
> - **Position-level selection** (`positionIds.length > 0`) — the sub-position draw path the "Selective" asset-guard variants exist for. Since those guards are not live (Section 24), this path has zero coverage from any angle.
> - **Fixed-amount allocation** (`useFixedAmount`) — only the portion-based branch was exercised.
> - **The surcharge mechanism** — `_averagePressure` , the `maxAcceptableSurchargeBps` rejection (`SurchargeTooHigh`), and the volume-decay window (`_checkAndRecordVolume`) — never triggered; the happy-path test's volume pressure was negligible.
> - **The value-conservation bound** (`completeBefore >= completeAfter` , `InvalidFundValue`) — never deliberately tripped.
> - **`minValueOutBps` /slippage rejection** specifically on the plan path.
> - A manager-signed plan with `fusdAmount == 0` but nonzero allocations — the exact scenario one of this function's own code comments names as an audit finding it closes.
>
> None of this means the mechanism is broken — it means this report's own evidence for it stops at nonce-replay and signature-expiry protection, and should not be read as broader coverage than that.

11 Guard validation (negative tests)

Per the scope correction in Section 2, the Uniswap-related fixes are tested at the validation layer only — confirming the guard correctly rejects bad input — since exercising them fully would require a volatile asset this fund does not hold.

#	TEST	RESULT	EVIDENCE
15	SlippageAccumulator factory-mixup fix (negative control)	PASS	The old, still-live instance reproduces the exact same revert it has thrown since deployment, confirming the bug is real; the new instance's success was independently captured during the guard-upgrade script's own run.
16	Multi-hop validation (M-12)	PASS	A swap path routed through an unsupported intermediate token is correctly rejected: `"Frgmnt: unsupported hop asset"`.
17	Zero output-minimum rejection (M-09)	PASS	`"Frgmnt: amountOutMinimum must be > 0"` — rejected at the guard layer before the router is ever reached.
18	Morpho market configuration live	PASS	New `MorphoBlueManager` reports exactly 10 configured markets for the live pool, matching the pre-upgrade count.

M-11 SELF-DEALING REJECTION — CITED, NOT RE-DERIVED

A meaningful fork-level reproduction of M-11's self-dealing rejection isn't possible against this specific live pool's real identity (its real manager is deliberately not the new `MorphoBlueManager`'s owner — that separation is the fix). The existing dedicated unit suite is the correct evidence for this property instead: `npx hardhat test test/MorphoBlueManager.test.ts` — **25 of 25 tests pass**, including all 4 dedicated self-dealing cases.

12 Multi-user isolation & fairness

Every prior test used a single account. This round uses three independent, separately-funded users (A, B, C — staked 7,000 / 3,000 / 15,000 USDC respectively) exiting through three different withdrawal paths, to prove per-user isolation rather than a single account's math working out in isolation.

#	TEST	RESULT	EVIDENCE
19	Harvest proportionality & principal isolation	PASS	User A harvests; users B and C's own **staked principal (sfUSD balance) is unchanged** by A's action — the one guarantee the contract actually makes. See the callout below for a nuance this test surfaced and corrected.
20	Three simultaneous exits, three different paths	PASS	A exits via plain `withdrawCashImmediate` (received 7,001.14 USDC), B via the queued request/finalize/claim flow (3,000.49 USDC), C via a signed attested withdrawal plan (100.02 USDC) — all three in the same test run, no interference between them.
21	Final reconciliation	PASS	25,000 USDC deposited across the three users reconciles against 25,004.08 USDC total accounted for across their positions and withdrawals (the small excess is real accrued yield) — no user received another user's funds.

A REAL NUANCE, CAUGHT BY THIS TEST AND CORRECTLY RESOLVED — NOT A BUG

This test's first draft assumed harvesting must leave every other staker's `pendingReward()` numerically unchanged, and initially read as a failure. Tracing it into `PoolLogic._accrueYield()` showed why: `harvest()` updates one shared, pool-wide `compoundedRewardIndex` that every staker's pending reward is computed from — this is the contract's existing, already-reviewed design (documented internally as FNA-13), not a migration defect. The test was corrected to assert the guarantee the contract actually makes — one user's harvest cannot touch another user's staked principal — which holds. Flagging this here because an auditor independently reading the raw pending-reward numbers before vs. after could reasonably ask the same question; this is the answer.

13 Automatic pro-rata withdrawal (asset guards)

Tests 9 and 10 (Section 8) manually call Morpho/Aave `supply` / `withdraw` through `execTransaction`, exercising the contract guards that validate that dispatch. This is a different code path from what happens automatically, every time, whenever a user calls a plain withdrawal function and the pool happens to hold a lending position: `WithdrawalPlanLib`'s pro-rata loop calls each asset guard's own `getBalance` / `withdrawProcessing` directly. That path had not been exercised until this round.

#	TEST	RESULT	EVIDENCE
22	Automatic pro-rata pull from live Morpho + Aave positions	PASS	Established persistent positions (15,000 USDC into Morpho, 10,000 into Aave), then had a separate user withdraw more than the pool's idle USDC alone could cover. A plain `withdrawCashImmediate` call — no manual guard interaction — correctly pulled from both: `MorphoBlueAssetGuard.getBalance` dropped from $89,690.95 to $86,875.73; the Aave aToken balance dropped from 9999.999999 to 9686.375571. The user received the correct total.

14 ERC20Guard validation

#	TEST	RESULT	EVIDENCE
23	`approve()` dispatch validation	PASS	Approving a registered, supported asset (USDC, to Morpho) via `execTransaction` succeeds. The identical call against an asset with no registered guard at all (DAI, in this pool's current configuration) is correctly rejected — reverts with `InvalidGuard()` (selector `0x15e33ca3`, confirmed matched). Precisely: this is `PoolTxExecutor`'s own guard-registration check firing before dispatch, one layer earlier than `ERC20Guard`'s own internal logic — a more fundamental safety net, not a weaker result.

15 New protections introduced by this upgrade

This upgrade's own batch calls three new, protective initializers. Earlier rounds confirmed each call succeeded; this round confirms each one actually does something — has real teeth, not just a clean call.

#	TEST	RESULT	EVIDENCE
24	Deposit cap enforcement (`TokenLogic.maxDepositFusdSupply`)	PASS	Confirmed the real cap reads 500,000.0 fUSD against real outstanding supply of 124,863.97 fUSD. To force an over-cap attempt without needing unrealistic whale capital, the cap was temporarily tightened (via the real admin setter, DAO Safe) to outstanding+2,000 fUSD on an isolated snapshot: a 5,000 USDC deposit attempt correctly reverted `"TokenLogic: deposit cap exceeded"` ; a 500 USDC deposit right after still succeeded. Reverted afterward — the real 500,000 cap was never actually touched.
25	EUR/USD mode lock (FNA-40)	PASS	`eurUsdModeLocked()` reads `true` post-upgrade. The DAO Safe — AssetHandler's own owner — attempting `setEurUsdAggregator(...)` correctly reverts `"Frgmnt: eur/usd mode locked"` . Confirms the valuation basis cannot be switched to EUR after this upgrade, by anyone, including the owner.
26	Sequencer uptime feed (FNA-50)	PASS wiring confirmed, live revert not attempted	`sequencerUptimeFeed()` reads the correct, real Chainlink Base L2 feed address. Calling `getUSDPrice(USDC)` — which internally runs the sequencer-up check — executes successfully against that live feed's real current state, confirming the check is genuinely wired into the price path rather than dead code. Forcing an actual "sequencer down" revert was not attempted: doing so would require fabricating a fake feed, which would prove nothing about the real wiring this test already confirms. Stated plainly rather than stretched for a result.

16 Access control (unauthorized-caller rejection)

Four genuinely privileged functions, each confirmed to reject a freshly-generated, uninvolved account and to still accept its real, intended caller.

#	FUNCTION	UNAUTHORIZED CALLER	REAL CALLER
27a	`PoolLogic.finalizeCashWithdraw`	PASS reverts `OnlyManager()` (selector `0xc0b2335a` , confirmed matched)	PASS the real pool manager finalizing the same request succeeds
27b	`MorphoBlueManager.setPoolMarkets`	PASS reverts `OwnableUnauthorizedAccount(address)` (`0x118cdaa7`)	PASS the real owner (DAO Safe) succeeds
27c	`Governance.setContractGuard`	PASS reverts the same `OwnableUnauthorizedAccount`	PASS the real owner (DAO Safe) succeeds
27d	`PoolManagerLogic.setUserActionsRouter`	PASS reverts `"only factoryOwner allowed"`	PASS the real factoryOwner succeeds

A TEST-HARNESS BUG THIS ROUND CAUGHT IN ITSELF, WORTH RECORDING

The local test network has no fixed signer list configured — fresh test accounts are drawn from whatever the fork reports as unlocked, which grows every time an address is impersonated. An account picked by a high `getSigners()` index silently resolved to an address already impersonated earlier in the same run (the live pool manager, in this case) rather than a genuinely neutral one — which initially made one access-control test pass for the wrong reason. Fixed by drawing every fresh test identity from a freshly generated wallet instead of a signer-list index. Checked every earlier round's own account selection against this same risk directly: all of them stayed within Anvil's fixed set of 10 default accounts (indices 0–9, which can never collide with an impersonated address), so none of the prior 23 tests' results are affected — this was isolated to this round's own new tests before it reached anything already reported.

17 Reinitializer protection & upgrade-script idempotency

Two further checks, run after the access-control round: whether each of this upgrade's own mandatory initializers is protected against being called a second time, and — separately, a question about the tooling rather than the contracts — whether the upgrade scripts themselves are safe for an operator to accidentally run more than once.

Reinitializer protection (contract-level)

#	TEST	RESULT	EVIDENCE
28	`PoolLogic.initializeAutoCompounding()` re-run	PASS	Correctly reverts `InvalidInitialization()` (selector `0xf92ee8a9`, confirmed matched) — OpenZeppelin's `reinitializer(2)` guard, independently confirmed live, not assumed from the source.
29	`TokenLogic.initializeDepositFusdCap(...)` re-run	PASS	Same `InvalidInitialization()`, same selector, same mechanism.
30	`PoolLogic.initializeWithdrawalEscrow()` re-run	PASS	Reverts, but **not** via `reinitializer` — this one specific initializer uses a plain already-set flag check instead, confirmed by matching the actual revert selector to `EscrowAlreadySet()` (`0x8ee0acc0`), not `InvalidInitialization()`. Functionally equivalent (a second call cannot succeed either way), but a real, worth-knowing implementation difference for anyone reasoning about this contract's upgrade safety from the outside.
31	`PoolLogic.initializeAttestedWithdrawal(...)` re-run	PASS	Not one of this upgrade batch's own mandatory calls (it configures the attester this report sets up fresh in Section 10), but the same `reinitializer(3)` mechanism — consumed for real once in this test scope, then a second attempt with deliberately different parameters (different attester address, rotation delay, decay window, surcharge ceiling) correctly reverts `InvalidInitialization()`, confirming the attester and its risk parameters cannot be silently reconfigured post-launch.

WHY THIS MATTERS

All four initializers checked here — the three mandatory ones this upgrade's own batch calls, plus the attested-withdrawal initializer this report separately exercises in Section 10 — are confirmed non-replayable, by two different but equally effective mechanisms. A second, accidental execution of the core upgrade's Phase 2 batch cannot silently re-initialize any of this state, and neither can a second attempt to reconfigure the attester.

Upgrade-script re-run safety (tooling, not contract behavior)

Separately: nothing stops the same Safe batch JSON from being proposed twice by mistake, or a script being re-invoked with `SEND=1` more than once. This was tested directly, by actually running each script twice against the same fork state rather than reasoning about it abstractly.

REAL GAP FOUND — `UPGRADE_SOFTSTACK_GUARDS.TS` (FIXED)

This script had no awareness of its own prior runs. A second, careless run deployed an entirely new, third generation of every guard and silently re-registered it with `Governance` — with no error, no warning, and no on-chain revert of any kind, because `Governance.setContractGuard` / `setAssetGuard` overwrite unconditionally rather than checking whether the slot already holds something other than the default. The correct, already-live deployment from the first run would simply be orphaned, with the script's own console output looking exactly as clean as a genuine first run. Confirmed by actually running the script twice against the same fork state. **Fixed**: the script now reads Governance's current registration for all four guard slots it manages at the very start of Preflight and compares each against its known pre-upgrade address; if any slot has already moved away from that baseline, the script refuses to proceed, printing a clear per-slot explanation, and requires an explicit `FORCE_REDEPLOY=1` to intentionally create a new generation anyway. Re-verified end to end on the fork: run 1 (clean state) behaves exactly as before and proceeds; an unmodified run 2 against the now-migrated state refuses immediately at Preflight, before deploying anything; run 3 with `FORCE_REDEPLOY=1` set proceeds, with the override explicitly logged.

SAME QUESTION, `UPGRADE_SOFTSTACK_CORE_CONTRACTS.TS` — LOWER SEVERITY, DOCUMENTED RATHER THAN CODE-CHANGED

This script already routes every mandatory initializer through a `reinitializer()`-gated path (see above), so a careless second `SEND=1` run cannot silently succeed: it wastefully re-upgrades up to two of the four proxies to the same implementation (harmless, if wasteful) before **loudly crashing** with `InvalidInitialization()` the moment it reaches the first already-initialized call. Unlike the guards script, there is no scenario where a re-run completes looking like a clean success while actually having silently orphaned anything — so no override flag was needed here. This behavior, and why it's acceptable as-is, is now documented directly in the script's own header comment rather than papered over with an unnecessary state check.

Full pre-existing regression-suite confirmation

Beyond this report's own targeted scenarios: the repository's full, pre-existing test suite was run clean and unmodified, directly against `feature/09-softstack-stage2-fixes` (the standard Hardhat unit-test network, not the Base fork) — **1,366 passing, 1 pending, 2 failing** (8 minutes). The 2 failures were confirmed, by reading the actual assertion and timing logic rather than assumed, to be the same pre-existing, timing-sensitive `test/TokenLogic.test.ts` flakes ("FNA-55: split mints cannot produce an earlier cooldown expiry than one equivalent mint") observed independently at the very start of this engagement, before any of this report's own work began. Zero new regressions from anything in this report.

18 Stablecoin-only Uniswap V3 testing & the artificial Aave debt scenario

Sections 8 and 11 previously tested the Uniswap-related guards only at the validation layer, since a real swap or LP mint appeared to require a volatile asset (WETH) outside this fund's stablecoin-only strategy. That constraint turned out to be avoidable: **USDC/USDT_BASE is a real, live, independently-verified stablecoin pair on Base** — the live `UniswapV3Factory` was queried directly for `getPool(USDC, USDT_BASE, fee)` across every standard fee tier, and the 0.01% pool at `0xD56da2B74bA826f19015E6B7Dd9Dae1903E85DA1` has by far the deepest real liquidity (~$112k/$113k at the time this was checked) — not a third-party guess. `USDT_BASE` (`0xfde4C96c8593536E31F229EA8f37b2ADa2699bb2`) is Basescan's "Bridged Tether USD" — verified source-matched, but explicitly **not issued, redeemed or backed by Tether itself** (Basescan's own token page says so); it is nonetheless the real, circulating token Base DEXs treat as USDT, and this exact address is already a hardcoded constant in this repo's own `AaveLendingPoolAssetGuard.sol` . It was registered as a second Frgmnt-supported asset using the protocol's own existing, already-deployed `USDPriceAggregator` (a fixed $1.00 Chainlink-compatible stub, already used for USDC itself via `scripts/add_assets.ts`) — no genuine Chainlink USDT/USD feed on Base could be confirmed, so this choice is disclosed plainly rather than silently treated as equivalent to a real market feed. This unblocked a real swap and the full Uniswap V3 LP-minting lifecycle, never exercised at all before this round, plus a deliberately artificial Aave debt scenario to exercise M-08's one remaining untested branch.

Real Uniswap V3 swap & SlippageAccumulator bound enforcement

#	TEST	RESULT	EVIDENCE
32	Real swap through the new `UniswapV3RouterGuard`, happy path	PASS	Swapped 1,000 USDC for 1,001.12 USDT_BASE via the real 0.01% pool through a genuine `execTransaction` dispatch — the real happy-path surface M-09/M-12 were only ever tested as rejections before.
33	`SlippageAccumulator`'s cumulative cap actually rejects a swap once crossed	PASS	A first escalating-size approach found the organic cost unmeasurable at affordable sizes: an $8,000 swap against this $112k-deep pool showed **0** accumulated slippage, because USDT_BASE's fixed $1.00 price and USDC's real (sub-$1.00) Chainlink price happen to net out any small real cost in this specific direction (see the correction callout below). Rather than chase an unrealistic trade size against a real, finite pool, this used the same pattern already reported for the deposit-cap test (Section 15): the real cap was temporarily tightened from 50,000 to 1 via the live owner-gated `setMaxCumulativeSlippage()`, then an identical-size swap in the reverse direction (USDT_BASE → USDC, which carries a real, positive measured cost under this same accounting) correctly reverted: `"slippage impact exceeded"`. Real cap restored afterward via snapshot, never left tightened.

A REAL ACCOUNTING NUANCE THIS TEST SURFACED, WORTH KNOWING

`SlippageAccumulator` values both sides of a swap by USD price, not by the pool's own execution quality — so registering USDT_BASE against a fixed $1.00 aggregator means a USDC→USDT_BASE swap's measured "cost" is partly an artifact of the gap between USDC's real (sub-$1.00) Chainlink price and that fixed assumption, not purely real AMM slippage. This only affects this report's own test design choice (a real Chainlink USDT/USD feed on Base could not be confirmed to exist); it says nothing about the guard's correctness, which Test 33 still proves directly by forcing a real rejection.

UniswapV3NonfungiblePositionGuard (M-14) — full lifecycle, never exercised before this round

#	TEST	RESULT	EVIDENCE
34	Real LP mint	PASS	Minted a real USDC/USDT_BASE position (tick range [-9, 31] around the pool's then-current tick 11); M-14's `assertFairPrice` and mint-loss-within-tolerance checks both passed for a real, honest mint; confirmed tracked in `NftTrackerStorage`.
35	`increaseLiquidity` on the same position	PASS	Added 800 USDC / USDT_BASE each to the open position — no revert, confirming M-14's shared loss check also passes on this path.
36	`decreaseLiquidity` + `collect` (partial close)	PASS	Liquidity halved, principal + fees collected to the pool; position left open and still tracked.
37	FNA-48: removal blocked while the position is tracked	PASS	Attempting to delist the Uniswap position manager itself, and separately USDT_BASE specifically, both correctly reverted while the position remained open. The USDT_BASE case is blocked by `removeTokenCheck()` (FNA-48) and/or the ordinary nonzero-idle-balance check — not isolated from each other in this test, since both run in the same removal path; delisting is confirmed blocked either way.
38	`burn` (full close)	PASS	Position fully closed; confirmed no longer tracked, which would now allow the FNA-48 removal checks above to pass if actually attempted.

Test 39 — M-08's debt-gated branch, exercised for real via an artificial Aave position

M-08 (Section 8) remained the one guard fix never exercised beyond a storage/bytecode presence check, since this fund's strategy never borrows. This test deliberately constructs an artificial debt position, by calling Aave directly and bypassing Frgmnt's own guard dispatch on purpose (that dispatch is exactly what normally prevents the fund from reaching this state), specifically to prove the fix.

> **A REAL, EXTERNAL FINDING, FOUND WHILE BUILDING THIS ONE TEST — NOT A CONTRACT BUG**
>
> Checked directly via `getReserveConfigurationData`: **every stablecoin reserve on the real, live Aave V3 Base deployment — USDC, USDT_BASE, GHO, and EURC — currently has an LTV of 0%.** This is Aave's own governance-set risk parameter, not a Frgmnt or test artifact, and it is unconditional: a 0-LTV asset cannot even be enabled as collateral (`setUserUseReserveAsCollateral` itself reverts `UserHasAssetWithZeroLtv()` for it). Only volatile reserves (WETH, cbBTC) currently carry a nonzero LTV (81%) on Base. No all-stablecoin "collateral + debt" scenario is constructible on this protocol integration today, full stop, regardless of test design — a real, worth-knowing operational fact: this fund's real $109k+ Aave USDC position (Section 8's correction) currently has **zero borrowing power on Aave itself**, an extra real-world safety factor beyond anything Frgmnt's own guards provide. This test therefore uses WETH only as the enabling collateral — wrapped from fresh ETH directly on the fork, never touching this fund's real strategy or any real token holder — while the debt asset itself, the thing M-08's fix actually inspects, stays a stablecoin (USDC). The narrowest deviation a real external constraint allowed, not a design choice.

#	TEST	RESULT	EVIDENCE
39	OLD guard reverts, NEW guard degrades to 0	PASS	With a real, freshly-created Aave debt position (1,000 USDC borrowed against freshly-supplied WETH collateral) and no settlement swap route configured: the OLD, still-live `AaveV3LendingPoolAssetGuard.getNetRealizableBalance()` correctly **reverts**, exactly as the bug predicts. The NEW, post-upgrade instance **degrades gracefully to 0** instead — the fix working as intended, confirmed side by side on the same real position, not asserted from the source alone.

19 A second independent finding: NftTrackerStorage misconfiguration

Building Test 34 (the real LP mint) surfaced a second genuine, pre-existing bug — found by debugging a real failure via `debug_traceCall`, not assumed. It is unrelated to anything in the SoftStack Stage 1/2 batch and was missed by every earlier round of this rehearsal until a real mint was actually attempted.

ALREADY CERTIK-FLAGGED, NEVER ACTUALLY APPLIED TO THE LIVE DEPLOYMENT

`NftTrackerStorage`'s own source code comment on its `setPoolFactory()` function says it directly: "Auditor-flagged (CertiK): deploy_contract_guard.ts previously passed PoolLogic's address here instead of PoolManagerLogic's — poolFactory must implement getContractGuard(), which only PoolManagerLogic does." A live, direct read confirmed this is exactly the live deployment's current state: `NftTrackerStorage.poolFactory()` == `POOL_LOGIC_PROXY`, not `PoolManagerLogic`. The practical effect: every real mint/increaseLiquidity on this fund's Uniswap V3 integration calls `NftTrackerStorage.addData()` to register the new position, which authorizes itself via `IHasGuardInfo(poolFactory).getContractGuard(...)` — since `PoolLogic` has no such function, that call lands on `PoolLogic`'s own generic callback-accepting `fallback()` instead, which then rejects `NftTrackerStorage` as an unrecognized caller (`CallbackSenderNotAllowed()`). **The entire Uniswap V3 LP-minting feature — M-14 included — is non-functional on the live pool right now**, independent of guard correctness, and was until this rehearsal found and fixed it.

WORSE THAN A BAD VALUE: THE FIX FUNCTION ITSELF WAS NEVER DEPLOYED

The known remediation (`setPoolFactory()`, an `onlyOwner` escape hatch) already exists in this branch's source — but the live, deployed `NftTrackerStorage` implementation did not contain it at all (confirmed directly: selector `0x473597a0` is absent from the live implementation's own bytecode). The proxy itself had never been upgraded since that fix was written, so calling `setPoolFactory` was never even possible before this.

Fixed as a genuine new addition to `upgrade_softstack_guards.ts`, not just this report's test script: a new `NftTrackerStorage` implementation is deployed and the proxy upgraded via its own `ProxyAdmin.upgradeAndCall` (DAO Safe-signed, added to that same batch). `setPoolFactory(PoolManagerLogic)` itself needs a separate signature — `NftTrackerStorage.owner()` is the pool manager Safe, not the DAO Safe that owns its `ProxyAdmin` — so the script now writes a **second** Safe batch file (`softstack-guards-pool-manager-safe-<chainId>.json`) containing, in order: `setPoolFactory(PoolManagerLogic)`, then `transferOwnership(DAO_SAFE)` — consolidating a custody gap found alongside the bug (`NftTrackerStorage` was the only contract either upgrade script touches whose ownership had never been moved onto the DAO Safe, unlike `MorphoBlueManager`, `SlippageAccumulator` and `AaveV3LendingPoolAssetGuard` earlier in this engagement). Order matters: reversing it would leave the pool manager Safe unable to call `setPoolFactory` at all. Re-verified end to end: the full fix landed cleanly in this rehearsal's final run, and Test 34 onward now reads the corrected value and succeeds for real, not through any workaround in the test script itself.

20 Comprehensive roles & access-control map (post-upgrade)

Read live, directly, against the fully-upgraded fork — not assumed from an earlier snapshot or from this report's own prior rounds.

ROLE	CONTRACT	CURRENT HOLDER	STATUS
`owner()`	PoolLogic	DAO Safe	Consolidated
ProxyAdmin `owner()`	PoolLogic's ProxyAdmin	DAO Safe	Consolidated
`manager()`	PoolManagerLogic	Pool manager Safe	Separate by design
`trader()`	PoolManagerLogic	A plain EOA (`0x4D444F4a7e13d0Fd0c06eC301e6047Db62` — confirmed not a contract	Single-key custody — flagged below
`factoryOwner()`	PoolManagerLogic	DAO Safe	Consolidated
ProxyAdmin `owner()`	PoolManagerLogic's ProxyAdmin	DAO Safe	Consolidated
`owner()`	AssetHandler	DAO Safe	Consolidated
ProxyAdmin `owner()`	AssetHandler's ProxyAdmin	DAO Safe	Consolidated
`DEFAULT_ADMIN_ROLE`	TokenLogic	DAO Safe (confirmed via `hasRole`)	Consolidated
`owner()`	Governance	DAO Safe	Consolidated
`owner()`	NftTrackerStorage	Pool manager Safe (pre-fix)	Gap found this round — fixed in the same batch (Section 19)
ProxyAdmin `owner()`	NftTrackerStorage's ProxyAdmin	DAO Safe	Consolidated (was already correct — only the contract's own owner lagged)
`owner()`	SlippageAccumulator	DAO Safe (post-handover, this upgrade)	Consolidated
`owner()`	MorphoBlueManager	DAO Safe (post-handover, this upgrade)	Consolidated
`owner()`	AaveV3LendingPoolAssetGuard	DAO Safe (post-handover, this upgrade)	Consolidated
(none)	UniswapV3RouterGuard / UniswapV3NonfungiblePositionGuard / MorphoBlueContractGuard	Not `Ownable` — confirmed directly (calling `owner()` on each simply reverts)	Stateless validators, immutable constructor config only — no owner to assign
`withdrawalAttester()`	PoolLogic	`address(0)` — confirmed on the real, live deployment	Confirmed unset — the attested-withdrawal feature is not live

A ROLE THIS REPORT HAD NOT PREVIOUSLY TRACKED: `POOLMANAGERLOGIC.TRADER()`

Confirmed to be a plain externally-owned account, not a multisig or any other contract. The `trader` role carries real privileges (e.g. `changeAssets` when `traderAssetChangeDisabled` is false) and is a single-key custody point, structurally the same category of risk this engagement's own centralization-review work has flagged elsewhere for other roles. Not fixed as part of this upgrade — flagging it here for the team's own judgment on priority, scope, and timeline; this report does not recommend a specific remediation for it.

21 Role-scoped testing (user / trader / manager) & migration-safety verification

Requested directly: exhaustive testing of the user, trader, and manager function surfaces specifically, plus explicit verification that the upgrade itself cannot induce any loss of pending rewards or existing funds. The attested withdrawal plan (also asked about directly) was already covered in full in Section 10 — happy path, nonce-replay rejection, and expired-deadline rejection, all passing; not repeated here.

Migration safety: pending rewards and principal, verified bit-for-bit across the real upgrade

Section 7 already confirmed aggregate `totalFundValue` /fUSD supply preservation. This round goes further: a specific, real staking position's **individual pending reward** was tracked through the actual upgrade transition — the exact migration path this upgrade's `initializeAutoCompounding()` depends on — using the live contract's own real code paths (deposit + stake), not a synthetic shortcut. The public Base RPC rate-limits `eth_getLogs` too heavily to enumerate a real historical staker directly (confirmed by hitting that limit repeatedly, not assumed); this is the closest achievable substitute, and methodologically equivalent: it exercises the exact same `rewardDebt` -checkpointing code every real historical staker's own deposit already went through.

#	TEST	RESULT	EVIDENCE
40	Pending reward preserved exactly across the real upgrade	PASS	A test position staked 9,997.9 fUSD on the live, pre-upgrade contract; 6 hours of real Morpho/Aave yield were then recognized into `rewardPerShare` via a second staker's own stake (not the test position's own action, so nothing was self-consumed) — pendingReward: **0.349294113331494496 fUSD**. Immediately after running the real core-contracts upgrade (which calls `initializeAutoCompounding()`): pendingReward read back as **0.349294113331494496 fUSD** — identical to 18 decimal places. `autoCompoundStartRewardPerShare` was confirmed to have captured the exact pre-upgrade `rewardPerShare` value used in that computation, not assumed from the source.
41	Principal (sfUSD) and fUSD total supply unaffected by the upgrade itself	PASS	The same test position's sfUSD balance (9,997.9) was identical before and after. Total fUSD supply was identical to the last decimal before and after (129,560.469441949722674979) — the upgrade mints or burns nothing on its own. `totalFundValue` moved by ~$0.006 between the two reads, entirely attributable to the real blocks the upgrade transactions themselves took to mine — genuine additional yield, not a discrepancy.

NO PENDING CASH-WITHDRAW REQUESTS EXIST ON THE LIVE POOL RIGHT NOW — CONFIRMED, NOT ASSUMED

`PoolLogic.lastRequestId()` reads **0** on the real, live deployment. There is currently nothing in the queued-withdrawal pipeline for this upgrade to preserve or risk — stated plainly rather than fabricating a request to test against, which would prove nothing about the real, current state.

TRADER role — never previously exercised at all in this report

`PoolManagerLogic.trader()` (Section 20: a plain EOA) has two genuine privileges distinct from the manager's: `execTransaction` (via `PoolTxExecutor` 's own `OnlyManagerOrTrader` check) and `changeAssets` (via its own separate check, gated additionally by `traderAssetChangeDisabled` — confirmed `false` on the live pool, i.e. currently active). Every guard-dispatch test earlier in this report used the manager as caller; the trader was never tried.

#	TEST	RESULT	EVIDENCE
42	Trader-signed `execTransaction` — real Morpho supply + withdraw	PASS	The trader EOA supplied 1,000 USDC to a real Morpho market and withdrew it back, identical in substance to the manager-signed version tested earlier (Section 8) — confirms `PoolTxExecutor` genuinely accepts the trader, not only the manager.
43	Trader-signed `changeAssets` — add then remove a supported asset	PASS	The trader added USDT_BASE as a supported asset, then removed it again, both via real trader-signed calls — `isSupportedAsset` flipped `true` then back to `false` each time.
44	Trader rejected by manager-only functions	PASS	The same trader EOA calling `setPoolPrivate` , `setAllowedCallbackSender` , and `finalizeCashWithdraw` all correctly reverted — the trader's elevated access is real but still narrower than the manager's.

Plain USER — confirmed rejected by both manager- and trader-gated functions

#	TEST	RESULT	EVIDENCE
45	Random unrelated account rejected by `execTransaction` and `changeAssets`	PASS	A freshly-generated wallet with no role at all correctly reverts on both — confirms a plain depositor cannot reach either the manager's or the trader's privileged surface, only the ordinary user-facing functions already exhaustively tested in Sections 7, 9, 10, and 12.

MANAGER — additional functions beyond what earlier sections already covered

#	TEST	RESULT	EVIDENCE
46	`setPoolPrivate` (real call) + `changeManager` (access proven via staticCall only)	PASS	`setPoolPrivate` flipped the real flag and back, reverted via snapshot afterward — never left changed. `changeManager` was deliberately only staticCall'd, with the SAME manager address and an inert name string, specifically to prove the manager's own access without ever actually reassigning management on a rehearsal fork — succeeded, confirming the manager can call it.

Aave V4 / Morpho Vault V2 — confirmed out of scope, not silently skipped

Asked directly: this codebase contains substantial, separate integrations for Aave V4 (Spoke + Tokenization) and Morpho Vault V2, each with its own contract guards, asset guards, and managers. Checked live and directly, not assumed: `PoolManagerLogic.getSupportedAssets()` lists exactly four assets today (USDC, the Uniswap V3 position manager, the Aave V3 pool, Morpho Blue core), and `Governance.assetGuards()` only has types 1–4 registered. **Neither Aave V4 nor Morpho Vault V2 has any live exposure on this pool right now**, and neither upgrade script in this engagement touches them — there is no real position for this rehearsal to test, and no SoftStack finding in either integration to verify. Their own dedicated unit-test suites (9 files) were re-run directly to confirm they are not simply unmaintained: **307 of 307 tests pass**. If the team activates either integration before or after this upgrade, that is a separate, future configuration step deserving its own rehearsal — not something this report can test against a position that does not exist.

22 Manager fee-minting & fee-governance testing

Asked directly: never previously tested in this report. The live pool's `performanceFeeNumerator` is already 2000 (20%) — not zero — so every deposit/stake/harvest throughout this entire engagement has already been implicitly exercising the fee-mint path (`_accrueYield()` mints `performanceFee + managementFee` in fUSD to the manager on every such call); this round checks it explicitly and quantitatively for the first time, and exercises the full fee-increase governance flow for the first time.

#	TEST	RESULT	EVIDENCE
47	Real fee mint via `announceFeeIncrease` → wait the delay → `commitFeeIncrease`	PASS	A fresh depositor staked, real yield accrued for 6 hours, then the manager announced `performanceFeeNumerator` 2000→2500, waited exactly the announced delay, and committed. The manager's fUSD balance grew by precisely the same amount `totalPerformanceFee` 's own ledger grew by (2.230005200544 fUSD both) — `mintManagerFee()` mints exactly what the fee ledger records, not more or less. `performanceFeeNumerator` correctly reads 2500 afterward.
48	`renounceFeeIncrease` cancels a real pending announcement	PASS	After announcing, `announcedFeeIncreaseTimestamp` was nonzero; after renouncing, both it and `announcedPerformanceFeeNumerator` correctly reset to 0.
49	Fee-increase bounds (max per-cycle change, commit delay) correctly enforced	PASS	Announcing beyond `getMaximumPerformanceFeeChange()` (2000, i.e. 20 points per cycle) correctly reverted; committing immediately, before the announced delay elapsed, correctly reverted.
50	Fee-governance access control + the immediate-decrease path	PASS	A random unrelated account calling `announceFeeIncrease` / `commitFeeIncrease` / `renounceFeeIncrease` all correctly reverted. The manager's real, immediate `setFeeNumerator` decrease (no delay needed) succeeded; the manager attempting an increase through that same no-delay function correctly reverted — increases can only ever go through the delayed announce/commit path.

REAL FINDING — `COMMITFEEINCREASE()` WITH NOTHING ANNOUNCED SILENTLY ZEROES EVERY FEE INSTEAD OF REVERTING

Test 51 (below) found this directly, not from reading the source alone: calling `commitFeeIncrease()` when `announcedFeeIncreaseTimestamp` is 0 — either because nothing was ever announced, or because a prior announcement was renounced — does **not** revert. `require(block.timestamp >= announcedFeeIncreaseTimestamp)` is trivially satisfied when the right-hand side is 0, and nothing else checks that an announcement is actually pending before the function proceeds to call `mintManagerFee()` and then `_setFeeNumerator(0, 0, 0, 0)` — silently zeroing `performanceFeeNumerator` (and every other fee numerator) with no announcement, no delay, and no error. Confirmed byte-for-byte identical in both the `audit` branch and `feature/09-softstack-stage2-fixes` (diffed directly) — **this is a real, pre-existing gap already live on real mainnet today, completely unrelated to and unchanged by this SoftStack upgrade.** It is not a user-fund-safety issue — a lower fee only ever favors stakers, never harms them — but it is a genuine manager-revenue-integrity footgun: a manager who calls `commitFeeIncrease()` by mistake (e.g. after forgetting they already renounced, or confused by a UI) silently forfeits their own fee entitlement instead of getting a clear, actionable revert. Not fixed here — fixing it would mean changing `PoolManagerLogic.sol` 's own source, which is a scope decision for the team and a future audit round, not something this upgrade-rehearsal report should quietly patch on its own. A straightforward fix, for the team's own consideration: `require(announcedFeeIncreaseTimestamp != 0, "nothing announced")` at the top of `commitFeeIncrease()`.

#	TEST	RESULT	EVIDENCE
51	`commitFeeIncrease()` with nothing announced — does it revert, or silently zero fees?	**FAIL (real finding, see above)**	Did NOT revert. `performanceFeeNumerator` went from 2000 to 0, with no announcement, no delay, and no error. Re-run cleanly after an unrelated test-script mistake (an accidental duplicate call) was caught and fixed in the first attempt — the result was identical both times.

23 Self-review: fork fidelity & testing-completeness verdict

Asked directly, after the fee-governance finding above: is the fork itself trustworthy, and is this rehearsal's testing actually enough to validate the upgraded and newly-deployed contracts? Answered plainly, including where the honest answer is "not fully" rather than reaching for reassurance.

Fork fidelity — re-verified directly, not assumed

CHECK	RESULT
Chain ID	**PASS** — reads 8453 (real Base), confirmed live on the fork, not configured/assumed
Block number & timestamp sanity	**PASS** — latest block's timestamp decodes to a real, current date, consistent with a genuinely live fork rather than a stale or fabricated snapshot
USDC identity	**PASS** — name/symbol/decimals (“USD Coin”/USDC/6) and a real-scale total supply (~4.3B) match Circle's actual USDC on Base, not a mock
Live PoolLogic implementation address	**PASS** — read directly from the proxy's own EIP-1967 implementation slot; matches `docs/deployments.md` 's own documented address exactly, an independent cross-check against this project's existing records
Live implementation bytecode length	**PASS** — the real, on-chain deployed bytecode is exactly 47,856 hex characters; a fresh compile of the `audit` branch (checked out in a separate worktree, compiled independently) produces a `PoolLogic` artifact of the identical length
Storage layout, audit vs. feature/09	**PASS** — already tool-verified in Section 6 via `forge inspect storage-layout` against both branches, field-by-field, for all four core proxies — the check that actually matters for upgrade safety, independent of the bytecode-length check above
Governance/custody addresses	**PASS** — the DAO Safe address has real contract code (344 bytes, a genuine Gnosis Safe proxy), not an EOA mistakenly treated as a multisig

> **ONE HONEST LIMITATION FOUND TODAY, AND PRECISELY WHAT IT DOES AND DOESN'T AFFECT**
>
> The public Base RPC (`mainnet.base.org`) rate-limits `eth_getLogs` heavily enough that enumerating real historical token holders directly (attempted for Section 21's migration-safety test) failed even with chunking, retries, and backoff — this is why that test uses a simulated-but-real-code-path staker instead of a genuine historical one (disclosed there already). This limitation is specific to **log-history queries `eth_call` / `eth_sendTransaction` used throughout this entire engagement — hundreds of calls across more than fifteen full fork rehearsals today alone — succeeded without a single rate-limit failure, which is itself evidence the limitation doesn't extend to normal read/write operations. Not an unlimited RPC budget, but not a reliability concern for anything this report's own test results depend on.**

Testing-completeness verdict: what's covered, what isn't

51 tests across this report, all passing except the one real finding above, plus the repository's own 1,366-test regression suite and the 307-test Aave V4/Morpho V2 suite, both passing clean. That is substantial, real coverage — but "substantial" is not the same as "exhaustive," and this review's job is to say precisely where the line is, not to imply there isn't one.

COVERED WITH REAL, PASSING FORK EVIDENCE	COVERED, BUT WITH A NAMED, DISCLOSED LIMITATION	NOT COVERED BY THIS REHEARSAL AT ALL
Core user flows (deposit/stake/harvest/unstake/withdraw, all variants) Attested withdrawal plan (happy path, replay, expiry) Morpho Blue & Aave V3 supply/withdraw, both dispatch paths Uniswap V3 swap, SlippageAccumulator cap, full LP lifecycle Multi-user isolation, 3 simultaneous exit paths Deposit cap, EUR/USD lock, sequencer feed wiring Access control on 4+ privileged functions Reinitializer protection on 4 initializers Trader role (execTransaction, changeAssets), both positive and negative Manager fee-minting, fee-increase governance (announce/commit/renounce/decrease) Pending reward & principal preservation across the real upgrade Upgrade-script idempotency (guards script refuses a careless re-run) Storage layout (all 4 core proxies, tool-verified)	M-11 self-dealing rejection — cannot be triggered against this pool's real identity; cited the existing dedicated unit suite (25/25) instead (Section 11) M-08's debt-gated degrade path — real on real Aave V3 Base state is currently impossible for any stablecoin (every stablecoin reserve has 0% LTV, Section 18); tested via a disclosed, artificial WETH-collateral bypass instead Migration-safety test used a simulated-but-real-code-path staker, not a genuine historical one, due to the RPC log-query limitation above	The brief window between the DAO Safe's core-contracts batch landing and its guards batch landing — both are separate Safe transactions, not one atomic execution, so there is a real (likely short) interval where the new core contracts are live but the old guards are still registered. Reasoned through (the new core code does not depend on the new guards for its own correctness, and the old guards remain valid against the state they were designed for) but not empirically fork-tested as its own scenario. Aave V4 / Morpho Vault V2 functional behavior — confirmed zero live exposure (Section 21), so correctly out of scope for THIS upgrade, but genuinely untested by this rehearsal beyond citing their own separate unit suite. The pre-existing, documented "pool factory mixup" availability bug and any other previously-flagged-but-not-yet-remediated findings outside this upgrade's own scope (tracked separately, not re-verified here).

> **VERDICT**
>
> For what this upgrade itself changes — the four core proxy implementations, the six redeployed guards, and the fifth proxy upgrade (NftTrackerStorage) — the evidence is strong enough to validate the upgrade as safe to propose. Storage layout is tool-verified, not assumed. Fund value, fUSD supply, and a real staking position's individual pending reward are all confirmed preserved to the exact decimal across the real upgrade transition. Every guard this upgrade touches has been exercised through a real, passing dispatch — manager, trader, and (where applicable) a correctly-rejected plain user. The one real new finding this round (the fee-commit gap) is independently confirmed to be unrelated to and unchanged by this upgrade — it does not block proposing the three Safe batches already assembled, but it is a genuine, separate item the team should decide how and when to fix, and is now recorded in Section 27. Nothing in this rehearsal's own results should be read as a claim that every line of this codebase has been exercised — the table above says plainly which parts haven't, and why.

24 Systematic guard-by-guard audit, plus the three dormant guard families proven against real infrastructure

Asked directly: test every guard in the codebase individually, not just the ones already covered by earlier rounds. The full inventory — 12 asset guards, 12 contract guards/managers — was enumerated and each one's live status checked directly rather than assumed from its filename or docs label.

Confirmed dormant, correctly out of scope

Cross-checked two independent ways: `docs/deployments.md` 's own Asset Guards and Contract Guards tables list only Morpho Blue (type 1), Aave V3 (type 2), Uniswap V3 (type 3), ERC20 (type 4) as asset guards, and only `AaveLendingPoolGuardV3` , `MorphoBlueContractGuard` , `MorphoBlueManager` , `MorphoBlueRewardClaimGuard` , `UniswapV3RouterGuard` , `UniswapV3NonfungiblePositionGuard` as contract guards — nothing for Aave V4 Spoke/Tokenization or Morpho Vault V2. Separately confirmed live on the fork: calling `getOwnedPositionIds()` directly against the registered Aave asset guard address reverts, proving the base `AaveV3LendingPoolAssetGuard` is what's actually registered, not the newer `AaveV3LendingPoolSelectiveAssetGuard` variant (consistent with `PoolLogic.withdrawalAttester() == address(0)` — the attested-withdrawal feature these "Selective" guards support isn't live either).

GUARD	STATUS
`AaveV3LendingPoolSelectiveAssetGuard` , `UniswapV3SelectiveAssetGuard` , `MorphoBlueLendingPoolSelectiveAssetGuard`	**Confirmed not live** — support the dormant attested-withdrawal sub-position feature; base variants are registered instead
`AaveV4SpokeAssetGuard` , `AaveV4SpokeSelectiveAssetGuard` , `AaveV4TokenizationAssetGuard` , `AaveV4SpokeContractGuard` , `AaveV4SpokeManager` , `AaveV4TokenizationContractGuard` , `AaveV4TokenizationManager`	**Confirmed not live** — not in `docs/deployments.md` , no live Governance registration; already-closed findings (FNA-08/09/44 etc.) exist as merged source, not live exposure
`MorphoVaultV2AssetGuard` , `MorphoVaultV2ContractGuard` , `MorphoVaultV2Manager`	**Confirmed not live** — same basis as above
`ClosedAssetGuard`	**Abstract base** — not independently deployable; exercised indirectly through its live concrete subclasses already under test (`AaveV3LendingPoolAssetGuard` etc.)

REAL FINDING — `UNISWAPV3ASSETGUARD` (ASSET TYPE 3) CARRIES THREE REAL, CLOSED FIXES NEITHER UPGRADE SCRIPT REDEPLOYED

Found by direct branch diff (204 diff lines), not assumed from the guard list alone. This asset guard — distinct from `UniswapV3NonfungiblePositionGuard` , the contract guard already correctly redeployed for M-14 — values every open Uniswap V3 LP position in NAV and gates the withdrawal-construction output floor. Three SoftStack/CertiK findings live in this one file:

- M-06. Splits what was previously one shared value with opposing pressure on each use into two independent parameters: `withdrawalSlippageBps` (the LP-output floor on `decreaseLiquidity` , unchanged default 100 = 1%) and a new `withdrawalTwapDeviationBps` (the spot-vs-TWAP oracle-manipulation ceiling itself, default 100 = 1%).
- FNA-16. A new per-pool `minimumPoolLiquidity` floor: the TWAP sanity-check now also rejects an under-liquid pool's observation outright (default 0 = disabled per pool until an admin tunes it).
- FNA-58 / FNA-37. `getBalance()` now values open LP positions at Chainlink-derived fair price instead of raw pool spot, and a single position whose pool has moved outside the fair-price band degrades to zero contribution (`isValuationComplete() == false`) instead of reverting the entire NAV read — the OLD guard's `getBalance()` calls the reverting `assertFairPrice()` directly, so one manipulated position there freezes `totalFundValue()` , stake/unstake/harvest, and immediate withdrawals for the whole pool.

Live config already matched the new contract's own hardcoded defaults exactly (`withdrawalSlippageBps=100` , `withdrawalTwapWindow=600`) — nothing to replay. The same stale-custody gap already fixed for three other guards this engagement was present here too (`admin()` still the retired `GOVERNANCE_SAFE` EOA, via a third, bespoke, non- `Ownable` pattern) — closed in the same deploy.

Fixed in `upgrade_softstack_guards.ts` : a new deploy block, an `admin` handover to the DAO Safe, an idempotency-preflight entry, and a new `Governance.setAssetGuard(3, …)` transaction added to the same DAO Safe batch. Re-run end to end on a fresh fork alongside the core-contracts script — deploy, handover, and registration all succeeded, and the full existing 39-test `functional_tests.ts` suite re-passed clean, including Round 6's own LP tests (34–38), now genuinely exercising the fixed guard for the first time rather than the stale one they'd been running against the entire engagement.

New targeted tests for the fix (`guard_hardening_tests.ts` , plus a follow-up isolation test)

Six new real fork tests, purpose-built for M-06/FNA-16/FNA-58, using a freshly minted, wide-range USDC/USDT_BASE LP position and self-calibrating price-pushes (binary search over real swap sizes, via snapshot/revert probes, rather than a guessed fixed amount against this pool's own, sharply nonlinear liquidity profile).

#	TEST	RESULT	EVIDENCE
52	FNA-58/FNA-37: NEW guard degrades gracefully where OLD guard reverts the whole NAV read	PASS	With the identical real position pushed ~124 ticks out of the Chainlink-fair band: OLD (still-deployed, no-longer-registered) guard's `getBalance()` correctly reverts (`Uni v3 LP price mismatch`) — reproducing the pre-fix bug live. NEW guard instead returns $0 for just this position with `isValuationComplete()=false` , leaving the rest of NAV readable.
53	M-06: a deliberately manipulated spot price blocks withdrawal construction	PASS	Reverted — but via `IncompleteNAV()` (PoolManagerLogic's own fund-level NAV-completeness gate, the same FNA-37 mechanism proven in Test 52) rather than M-06's own `Spot deviation too high` string specifically. Investigated further and resolved below (Test 57) rather than left as an open question.
54	M-06/FNA-37: withdrawal recovers after a real reverse trade restores spot	PASS	A genuine reverse-direction swap (selling the manipulated USDT_BASE back into USDC, restoring pool spot near the Chainlink fair price) plus advancing time past the TWAP window — the identical withdrawal then succeeds again. Confirms neither gate latches permanently closed; pool spot does not self-heal with time alone, only a real trade moves it, which is what this test actually does rather than just waiting.
55	FNA-16: withdrawal construction rejects an under-liquid pool once a floor is configured	PASS	`minimumPoolLiquidity` set (via the guard's real admin setter) to 1,000,000× the pool's own real `liquidity()` reading — deliberately unreachable. Withdrawal correctly reverts with the guard's own exact string, `UniswapV3AssetGuard: TWAP liquidity too low` .
56	FNA-16: withdrawal recovers once the floor is reset to 0	PASS	Resetting the floor back to 0 via the same real setter immediately restores the identical withdrawal — confirms the floor is live, reversible, per-pool config, not a one-way lockout.

57	M-06 isolation: proves the gate is independently reachable, not shadowed dead code (`m06_isolation_test.ts`)	PASS	Manipulate spot hard (Δ≈24,575 ticks) → advance 300s (half the 600s TWAP window, no further trades, so the TWAP blends roughly 50% pre-manipulation / 50% manipulated) → reverse-swap (binary-searched, not guessed) current spot back to within ~11 ticks of the original tick. `isValuationComplete()` reads `true` (current spot is back inside the Chainlink fair-price band — the Test 53 gate would NOT fire now). The withdrawal still reverts — this time with M-06's own exact string, `UniswapV3AssetGuard: Spot deviation too high` , not `IncompleteNAV()` . Confirms M-06 is live, necessary code for this specific scenario (a recent spike already arbitraged back to fair, where the TWAP feeding `_calcDecreaseLiquidity` 's own slippage math is still contaminated) — not redundant behind the broader NAV-completeness gate, as Test 53 alone left open.

TEST 53'S OPEN QUESTION, RESOLVED

Asked directly afterward: is M-06's own check ever actually reachable through any withdrawal path, or does the broader `IncompleteNAV()` gate always fire first and make it dead code? Worked out analytically first — on this pool (0.01% fee, tick spacing 1), M-06's 1% price-deviation threshold and the fair-price band's 0.5% sqrtPrice threshold both correspond to almost exactly the same ~100-tick deviation, which is why Test 53's single-instant manipulation couldn't separate them: both are measuring (effectively) the same magnitude at that instant, just against different reference points — TWAP for M-06, the Chainlink oracle for the broader gate. But those two reference points don't have to move together: a manipulation that's already been partly absorbed into the TWAP, then reversed back toward the oracle price, decouples them — current spot can be back near fair (the broader gate's own check passes) while the TWAP is still dragged toward the stale manipulated tick (M-06's own check still fails). Test 57 constructs exactly that sequence for real and gets M-06's specific revert string, settling it: the fix is independently necessary, not shadowed.

A SELF-INFLICTED TEST-HARNESS PITFALL, CAUGHT BEFORE IT REACHED A RESULT — NOT A CONTRACT ISSUE

An early version of this round's own setup funded the pool via a plain `transfer()` (free USDC, no matching FUSD liability) rather than a real deposit. That untracked "donation" inflated `totalFundValue()` far past `fusdTotalSupply()` ; the next `checkpointFeesForDeposit()` (any deposit anywhere triggers one) then computed a real management fee off that fabricated surplus — `netYield` 's own clamp (FNA-22) limited it to that surplus, but the surplus itself was already enormous, minting a six-figure FUSD fee in one shot and permanently blowing the real 500,000 deposit cap for the rest of that run. Confirmed directly (isolated by checkpointing standalone and watching `protocolFusdOutstanding` jump from ~119k to ~819k with zero elapsed time) before concluding anything about the contracts themselves. Fixed by funding through a real deposit instead — USDC in and FUSD out stay 1:1, so NAV and liabilities move together and never produce a phantom surplus. Not a contract bug: a genuine donation-style transfer of this fund's real USDC has never happened on live mainnet and isn't a code path any real user flow produces.

FOUND, DOCUMENTED, DELIBERATELY NOT IMPLEMENTED — `MERKLREWARDCLAIMGUARD` (FNA-19)

Confirmed via direct source diff: this contract guard is a pure rename of the `audit` branch's `MorphoBlueRewardClaimGuard` — selector check, single-caller-must-be-pool check, and claim-dispatch logic are byte-for-byte identical; only the contract/file name, comments, error strings, event name, and the `TransactionType` enum member's name changed. The enum's numeric value is explicitly unchanged (confirmed via the code's own inline comment), broadening the guard's documented scope from "Morpho rewards only" to any Merkl-sourced reward stream (Morpho Blue and Aave V4 Spoke). This guard is live and documented in `docs/deployments.md` (as "Morpho Rewards", address `0x0F61…Bb2Af`, confirmed to have real deployed bytecode) — and neither upgrade script redeploys or re-registers it, a third real, missed step.

Deliberately left out of the upgrade scripts this round. The real on-chain registration target — the Merkl Distributor contract address on Base specifically — could not be conclusively confirmed. Web research found only a general claim (Merkl's own docs and Morpho's integration docs both describe one Distributor address as used "on the vast majority of EVM chains" / "the same address across all supported chains") without an explicit, Base-specific confirmation. A direct on-chain check of the most likely candidate address found real deployed code at it, but `Governance.contractGuards()` is not registered against that specific address — so it is not the live target. A reverse lookup via `Governance`'s own `ContractGuardSet` event log, filtered by the known guard address, was attempted to find the real target directly but failed completely: the public Base RPC's `eth_getLogs` rate-limiting (already disclosed in Section 23) defeated every chunking/retry strategy tried across the full ~53M-block range. Rather than register a redeploy against an unconfirmed — and possibly wrong — target, this finding is recorded here for the team to confirm the real Distributor address (e.g. via an indexer with full log history, or directly from the team's own Merkl integration records) before adding the redeploy to a future batch. Implementing a guess here could silently misdirect reward claims, which is worse than leaving the gap open and disclosed.

Final guard inventory, as of this round

GUARD / MANAGER	STATUS
`AaveLendingPoolGuardV3`, `AaveV3LendingPoolAssetGuard` (M-08)	Live, tested, redeployed
`MorphoBlueContractGuard` (M-11), `MorphoBlueManager`, `MorphoBlueLendingPoolAssetGuard`	Live, tested, redeployed
`UniswapV3RouterGuard` (M-09/M-12), `UniswapV3NonfungiblePositionGuard` (M-14)	Live, tested, redeployed
`UniswapV3AssetGuard` (M-06, FNA-16, FNA-58)	Live, found this round, fixed & tested this round
`ERC20Guard`, `ClosedAssetGuard` (abstract)	Live / exercised indirectly, no redeploy needed
`MerklRewardClaimGuard` (FNA-19)	Live, found this round, documented only — real registration target unconfirmed, see above
Aave V4 Spoke (×3: Manager, ContractGuard, AssetGuard)	Not live on this pool — but now proven functionally correct against real Aave V4 Base infrastructure, see below
Morpho Vault V2 (×3: Manager, ContractGuard, AssetGuard)	Not live on this pool — but now proven functionally correct against a real Morpho Vault V2, see below
Aave V4 Tokenization (×2), the 3 "Selective" variants	Confirmed not live, not independently tested this round — correctly out of scope

Addendum: the dormant Aave V4 Spoke, Aave V4 Tokenization, and Morpho Vault V2 guards, actually tested against real forked infrastructure

"Confirmed not live" (above) means these guards have no live Governance registration on this pool today — true, and correctly out of scope for an upgrade-safety rehearsal. It does not mean the guards work. Asked directly to close that gap: deploy them fresh and register them against Governance on this fork only, then link them to the real, live external protocols they're built for — not mocks, not a synthetic fixture — and prove supply/deposit, automatic pro-rata withdrawal, manual withdrawal, and the critical security checks all actually work.

REAL, SOURCED ADDRESSES — NOT GUESSED, AND ONE WRONG CANDIDATE CAUGHT BEFORE USE

Aave V4 is live on Base (confirmed via Aave's own official address-book registry, `github.com/aave/aave-address-book`, module `AaveV4Base`) — a Hub-and-Spoke RWA deployment, not the fund's own stablecoin-lending model. Read `MAG7_SPOKE`'s real reserves directly on the fork: 9 reserves, 7 tokenized-equity collateral assets plus USDC at reserveId 7 (paused=false, frozen=false, borrowable=true — confirmed supply is not gated behind the "borrowable" flag, which governs the borrow side only, by reading Aave's real `ISpoke.sol` source from `github.com/aave/aave-v4` directly). The same registry's `TOKENIZATION_SPOKE` (a standard ERC-4626 vault wrapping USDC directly into the same Hub, assetId 7) was confirmed live and usable too (`maxDeposit` = 1,000,000 USDC). For Morpho Vault V2, a third-party developer blog's cited Base factory address (`0xA1D94F746d...`) turned out to have zero deployed code on Base — it's the Ethereum mainnet factory from the same source file, misattributed to Base. Caught by checking directly on the fork before use, not assumed. The real Base factory (`0x450112550807...`) came from Morpho's own SDK source (`github.com/morpho-org/sdks`) and was verified to hold real code before any further work.

A REAL, IMPORTANT CORRECTION, FOUND MID-ROUND — THIS WHOLE FAMILY MUST BE TESTED AGAINST THE UPGRADED POOL, NOT THE LIVE ONE

The Aave V4 Tokenization test initially failed in an alarming way: a real depositor's automatic withdrawal burned 2,499.33 FUSD for real but delivered 0 USDC — not a revert, a silent zero-value payout. Traced directly rather than assumed: every guard in this family sets `isPreValuedAssetGuard()=true` so `PoolManagerLogic.assetValue()` treats its already-USD-scaled `getBalance()` / `getWithdrawableBalance()` figure as final, with no further price multiplication. But the live, pre-upgrade `PoolManagerLogic` has no concept of `IPreValuedAssetGuard` at all — it predates that feature entirely. Its fallback path re-prices the already-USD-scaled figure anyway: `price(1.00) * amount / 10**decimals`. For `AaveV4TokenizationAssetGuard`, whose `getDecimals()` correctly returns the real ERC-4626 share decimals (6, matching this specific TokenizationSpoke), that re-pricing inflates the figure by 10^{12} — enough to size the fund-wide withdrawal portion as effectively zero for every asset, pool-wide, while the FUSD burn itself (computed on a different, unaffected code path) still goes through for real.

This is not a contract bug — it is a real, direct consequence of testing a feature against a pool that predates the upgrade which introduces it. It also means the two already-reported "PASS" results for Aave V4 Spoke and Morpho Vault V2 needed re-checking, not just assuming they were fine because the numbers looked plausible: both were first run on the same pre-upgrade pool. Re-traced directly: `AaveV4SpokeAssetGuard.getDecimals()` is hardcoded to `18` (its registered "asset" is a non-transferable pseudo-position, not a real token), and this specific Morpho Vault V2 instance's own share decimals happen to also be 18 — both exactly cancel the same re-pricing bug by coincidence, which is why those two results came out numerically correct anyway. Re-ran all three families against the properly upgraded pool (`upgrade_softstack_core_contracts.ts`, `SEND=1`, run first) and got identical, correct results for Spoke and Vault V2, and the previously-broken result now fixed for Tokenization — all 15 tests below reflect that corrected run. All three scripts now check for this precondition directly (`PoolLogic.compoundedRewardIndex()` — reverts pre-upgrade, returns a real value post-upgrade, the same signal the upgrade script's own preflight already uses) and refuse to run against the wrong baseline again.

#	TEST	RESULT	EVIDENCE
58	AaveV4Spoke: real supply through GiverPositionManager into the live MAG7_SPOKE	PASS	Deployed `AaveV4SpokeManager` / `ContractGuard` / `AssetGuard` fresh, registered against Governance on the fork, whitelisted reserveId 7, authorized both PositionManagers via a real `Spoke.setUserPositionManager()` call (FNA-08) dispatched through the asset guard's own `txGuard`. Supplied 3,000 USDC through the real, live `GiverPositionManager` — pool's USDC dropped by exactly 3,000; `getBalance()` reported the position correctly against real Aave V4 Base state.
59	AaveV4Spoke: automatic pro-rata withdrawal (`withdrawCashImmediate`) pulls from the real Spoke position	PASS	A real depositor's ordinary withdrawal correctly reached `withdrawProcessing()` and called `ISpoke.withdraw()` directly (no PositionManager delegation needed for a pool withdrawing its own position, per FNA-15) — real USDC received, position correctly reduced ($2,999.19 → $2,940.04).
60	AaveV4Spoke: manual withdrawal via TakerPositionManager (`approveWithdraw` + `withdrawOnBehalfOf`)	PASS	Manager-dispatched `execTransaction` path received exactly the requested amount through the real, live `TakerPositionManager`.
61	AaveV4Spoke: supply to a non-whitelisted reserveId is rejected	PASS	Correctly reverted `AaveV4SpokeGuard: reserve not whitelisted` — the FNA-10 active-reserve allowlist enforces against a real, unauthorized equity reserve on the live Spoke.
62	AaveV4Spoke: `approveWithdraw(spender != pool)` is rejected	PASS	Correctly reverted `AaveV4SpokeGuard: spender != pool` — the single most important check in this contract, per its own documentation: Aave V4's withdraw sends funds to `msg.sender`, not `onBehalfOf`, so an unrestricted spender could otherwise drain the pool's real position directly.
63	MorphoVaultV2: real deposit through `execTransaction` into a freshly-created, real vault	PASS	Created a genuine Morpho Vault V2 (USDC) through the real, live Base factory, deployed/registered the dormant guard family fresh, whitelisted it, deposited 3,000 USDC — pool's USDC dropped correctly, the real vault minted real shares, `getBalance()` read them back correctly.
64	MorphoVaultV2: automatic pro-rata withdrawal pulls from the real vault position	PASS	Same automatic `withdrawCashImmediate` mechanism already proven for Morpho Blue/Aave V3/Aave V4 Spoke — correctly reached this vault's own `withdrawProcessing()` and received real USDC back ($2,999.19 → $2,939.85).
65	MorphoVaultV2: manual `withdraw()` via `execTransaction`	PASS	Received exactly the requested amount through the real vault's standard ERC-4626 `withdraw()`.

66	MorphoVaultV2: `deposit(assets, receiver != pool)` is rejected	PASS	Correctly reverted `MorphoVaultV2Guard: receiver != pool` — minted shares can never be redirected to an arbitrary address.
67	MorphoVaultV2: deposit into a non-whitelisted vault is rejected	PASS	A second real vault, created the same way but never whitelisted, correctly rejected a deposit attempt with `vault not enabled` — a manager cannot register or use an arbitrary, unvetted ERC-4626 contract.
68	AaveV4Tokenization: real deposit through `execTransaction` into the real, live TOKENIZATION_SPOKE	PASS	Deposited 3,000 USDC directly into the real, live `TOKENIZATION_SPOKE` (ERC-4626, 6-decimal shares) — pool's USDC dropped by exactly 3,000, the real vault minted real shares, `getBalance()` correctly reported $2,999.19 against real Aave V4 Hub state.
69	AaveV4Tokenization: automatic pro-rata withdrawal pulls from the real position	PASS	The finding above, now fixed by testing against the upgraded pool: received 2,499.999996 real USDC for 2,499.33 FUSD burned, position correctly reduced ($2,999.19 → $2,940.04) — not the silent $0 the pre-upgrade pool produced.
70	AaveV4Tokenization: manual `withdraw()` via `execTransaction`	PASS	Received exactly the requested 300 USDC through the real TokenizationSpoke's standard ERC-4626 `withdraw()`.
71	AaveV4Tokenization: `deposit(assets, receiver != pool)` is rejected	PASS	Correctly reverted `AaveV4TokenizationGuard: receiver != pool`.
72	AaveV4Tokenization: deposit into a non-whitelisted vault is rejected	PASS	Reused the real, live `MAG7_SPOKE` as the "unauthorized" target (a real Aave V4 contract, just never whitelisted for this purpose) — correctly rejected.

RESULT

All 15 of these new tests pass, against real, live, forked external protocol infrastructure — not mocks — and against the correct, upgraded baseline. All three previously-dormant guard families are now shown to be functionally correct, not merely "present in the codebase." This does not change their live status: none is registered on this pool's real Governance today, and nothing here argues for or against activating any of them — that remains a product decision for the team. What it closes is the gap between "confirmed absent" and "confirmed would work if activated," which the earlier round of this report had not addressed, plus a real methodology gap (testing against the wrong pre/post-upgrade baseline) this round found and fixed in its own tooling. Scripts: `scripts/fork/aave_v4_spoke_integration_test.ts`, `scripts/fork/aave_v4_tokenization_integration_test.ts`, `scripts/fork/morpho_vault_v2_integration_test.ts`.

25 Issues found and resolved during rehearsal

Every problem the rehearsal actually surfaced, and whether it reflects a real contract issue.

Real bugs in the upgrade scripts themselves (fixed)

- Wrong signer for owner-gated calls. An early version of both scripts attempted to send Safe-owned transactions from the plain deploy key. Caught by a clean `OwnableUnauthorizedAccount` revert; fixed by impersonating the real on-chain owner.

- Ownership-transfer sequencing. A freshly deployed contract is owned by its deployer, not the eventual intended owner — the guard script initially had this backwards. Restructured so the deployer hands ownership over immediately after deploying.
- An ethers.js array-encoding quirk replaying the live Morpho market list onto the new manager (a read-only `Result` object can't be passed directly into another call's array argument). Fixed with a plain array conversion.
- No re-run safety in the guard-redeployment script. Covered in full in Section 17 — the most significant finding of that round.

A real, independent, pre-existing bug on the live deployment itself (fixed)

- `NftTrackerStorage.poolFactory()` misconfigured, and the fix function never deployed. Covered in full in Section 19 — the most significant finding of the final round, and the only finding in this report that is neither a SoftStack fix nor a tooling issue, but an independent, already-CertiK-flagged bug this rehearsal found live and fixed.

Fork-testing artifacts, not contract bugs (isolated and documented)

- Gas under-estimation on deep call chains. Several functions (`harvest` , `unstake` , `withdrawCashImmediate` , the Morpho/Aave supply-withdraw paths, and — found in the final round — the direct Aave `borrow()` call in Test 39) traverse unusually long nested delegatecall chains or otherwise confuse ethers.js's automatic gas estimation under Anvil. Resolved with explicit, generous gas limits throughout — a test-tooling adjustment, not a contract change.
- Frozen oracle data vs. advancing fork time. Advancing the fork's clock to clear the 1-day exit cooldown, or to let real interest accrue, pushes `block.timestamp` past `AssetHandler` 's correctly-functioning 24-hour price-staleness check, since the fork's Chainlink data is frozen at the pinned block — real mainnet oracles update continuously, so this cannot happen there. Resolved by raising that timeout once, upfront, as an explicit test-harness-only step.
- Real-wall-clock deadlines go stale on a long-running fork. Found in the final round: Uniswap's `mint` / `increaseLiquidity` / `decreaseLiquidity` deadlines were computed as `Date.now()/1000 + 3600` , but by the time Round 6 runs, the fork's own `block.timestamp` (repeatedly advanced by earlier tests) can already be ahead of "real wall-clock time + 1 hour" — Uniswap's own "Transaction too old" check compares against `block.timestamp` , not real time. Fixed by computing every deadline from the fork's own current block timestamp instead.
- `TxFailed()` masks the real revert reason. `CallResultChecker` 's generic wrapper around guarded calls discards the underlying revert on failure. Diagnosing the Aave supply issue, and later the `NftTrackerStorage` finding above, both required a direct, un-guarded `eth_call` / `debug_traceCall` against the real target to surface the actual cause.
- Missing fUSD allowance on the attested path. `withdrawCashImmediateWithPlan` needs the same `approve(POOL_LOGIC, amount)` allowance as `withdrawCashImmediate` — easy to miss since the attested path's `burnFrom` call is buried inside `WithdrawalPlanLib` .
- Plan-construction tolerance caps. An initial attested-plan test used a 100%-portion allocation with `minValueOutBps: 200` , exceeding the protocol's 1% `MAX_MIN_VALUE_OUT_BPS` ceiling. Corrected to a precisely-computed fixed amount, matching the repo's own existing attested-withdrawal test reference pattern.
- Signer-index collision. Covered in Section 16's callout — a fresh test account drawn from a high `getSigners()` index could silently resolve to an address already impersonated earlier in the same run. Fixed by using freshly generated wallets for new test identities.
- Whale/pot sizing across a long, cumulative test run. Round 6's own USDC/USDT_BASE funding and swap/mint sizes were initially fixed absolute amounts, which failed once the shared test whale had already been drawn down by the ~31 tests preceding it in the same run. Fixed by sizing Round 6's funding dynamically off the whale's actual live balance, and every subsequent swap/mint amount as a fraction of that round's own funded pot rather than a second independent fixed amount.

Several smaller test-design-only issues (harvesting before any yield had accrued; a fresh depositor's cooldown resetting before queued withdrawal could run) were identified and corrected in the same pass — all were the test missing a step a real user interface would always include, not gaps in the contracts.

26 External findings

> **OPERATIONAL, NOT A CONTRACT ISSUE**
>
> Aave V3's own USDC reserve on Base was found frozen (`isFrozen = true` , confirmed via `getReserveConfigurationData`) at the block this rehearsal was originally pinned to. This is an Aave-governance action, entirely unrelated to this codebase or upgrade. Confirmed unfrozen again at a later, current block (`53298399`), and the fork was re-pinned there for the Aave-dependent rounds of this report. Worth knowing operationally: Aave V3 USDC supply on Base can be temporarily frozen by Aave's own governance independent of anything in this codebase — not a new risk this upgrade introduces, just a real external dependency worth the team's awareness.

27 Outstanding items before real execution

1. Team and auditor review of all three Safe batches, across two separate Safes. This report's evidence is from a rehearsal; the actual Safe Transaction Builder JSON files these scripts produce should be reviewed line-by-line before being proposed. `upgrade_softstack_core_contracts.ts` writes one DAO Safe batch; `upgrade_softstack_guards.ts` now writes two — a DAO Safe batch (guard registrations + the new `NftTrackerStorage` implementation upgrade) and a separate pool manager Safe batch (`softstack-guards-pool-manager-safe-<chainId>.json` — `setPoolFactory` then `transferOwnership` , Section 19).
2. Re-confirm live state immediately before proposing. Both scripts re-read custody and pending-state on every run — but re-run them fresh right before the real proposal, don't reuse this rehearsal's output.
3. Decide execution order: core batch, then the DAO Safe guard batch, then the pool manager Safe batch — in that order, not any other. The pool manager Safe batch's `setPoolFactory` call does not exist as a callable function until the DAO Safe's `NftTrackerStorage` implementation upgrade has landed; proposing it first will simply fail. Core vs. the DAO Safe guard batch remain independent and safe in either order, as before.
4. Re-verify the deposit cap decision. `NEW_DEPOSIT_FUSD_CAP` is still 500,000 fUSD — comfortably above current supply, but this is a product decision the script does not make on its own.
5. Decide the real attester, deliberately, before ever enabling the attested-withdrawal feature on mainnet. This report proves the mechanism works correctly and safely in isolation (replay and expiry protections hold); it does not and should not be read as a recommendation on if or when to actually enable it live, or who should hold that key. Separately confirmed this round: `PoolLogic.withdrawalAttester()` reads `address(0)` on the real, live deployment right now (Section 20) — the feature genuinely is not live.
6. If the guard-redeployment script is ever run more than once against real mainnet, it will now refuse by default once Phase 2 has actually landed (Section 17) — this is intentional; use `FORCE_REDEPLOY=1` only if a genuine follow-up redeploy is actually intended, and treat that as a deliberate, reviewed decision each time, not a routine override. This check does not currently cover the `NftTrackerStorage` fix added this round — a careless second run would wastefully redeploy another implementation and re-call `setPoolFactory` / `transferOwnership` with the same values (harmless no-op) and then loudly fail on the second `transferOwnership` attempt (the pool manager Safe no longer owns the contract after the first run) — not silent, but not covered by the existing guard either; treat as a known, lower-priority gap, same tolerance level already accepted for the core script.
7. The guard batch's Aave swap is now against a real, substantial live position, not an empty one. Section 8's correction (re-verified 2026-10-31) found the live pool currently holds ~$109k in Aave V3 — the large majority of the fund's TVL — not the "empty" position earlier notes assumed. Swapping `Governance` 's registered `AaveV3LendingPoolAssetGuard` does not move or touch the tokens themselves (`PoolLogic` holds the aTokens directly; the guard is a stateless validator/accounting layer, confirmed by reading how `getUserReserveData` is keyed), so this does not change the batch's safety — but it does mean this specific swap now governs real, substantial value from the moment it lands, where it previously would have governed nothing live. Re-confirm the real balance immediately before proposing (per item 2 above), and weigh this guard's swap with that in mind rather than as a no-live-impact formality.
8. The `PoolManagerLogic.trader()` role is a single, plain EOA (Section 20) — not fixed as part of this upgrade, flagged for the team's own judgment on priority and timeline.
9. Every stablecoin on the real Aave V3 Base deployment currently has 0% LTV (Section 18) — an external, Aave-governed fact, not something in this codebase's control, but worth the team's awareness: this fund's real Aave position currently has zero borrowing power on Aave itself, regardless of this fund's own guards.
10. `PoolManagerLogic.commitFeeIncrease()` silently zeroes every fee when nothing is announced, instead of reverting (Section 22) — a real, pre-existing gap, confirmed identical on both `audit` and `feature/09-softstack-stage2-fixes` , so unrelated to and unaffected by this upgrade. Not a user-fund-safety issue and not blocking for proposing the three batches above, but a genuine manager-revenue-integrity item the team should decide how and when to fix — a straightforward candidate is adding `require(announcedFeeIncreaseTimestamp != 0, "nothing announced")` to the top of that function, in a future, separate change.
11. The brief interval between the core batch and the guards batch landing (two separate Safe transactions, not one atomic execution) was reasoned through but not empirically fork-tested (Section 23) — the new core contracts do not depend on the new guards for their own correctness, and the old guards remain valid against the state they were designed for, but this specific intermediate window has not itself been exercised on the fork.
12. Confirm the real Merkl Distributor registration target before redeploying `MerklRewardClaimGuard` (FNA-19) (Section 24) — a real, live, pure-rename fix that neither upgrade script currently touches. The team should confirm the exact on-chain address this guard needs to be registered against (e.g. via an indexer with full log history, or the team's own Merkl integration records) and add the redeploy to a future batch; this report deliberately does not guess at it.

28 Reproducing this rehearsal

Everything in this report can be reproduced locally. No API keys or credentials beyond a public RPC endpoint are required.

`./scripts/fork/start-base-fork.sh <block>`	Starts the local Anvil fork, pinned to a chosen block
`SEND=1 npx hardhat run scripts/upgrade_softstack_core_contracts.ts --network baseFork`	Runs the core-contracts upgrade
`SEND=1 npx hardhat run scripts/upgrade_softstack_guards.ts --network baseFork`	Runs the guard redeployment, including the new `NftTrackerStorage` implementation upgrade and fix (set `FORCE_REDEPLOY=1` only to intentionally re-run against already-migrated state)
`npx hardhat run scripts/fork/functional_tests.ts --network baseFork`	Runs the full, 39-test functional suite against the upgraded fork
`npx hardhat run scripts/fork/migration_pre.ts --network baseFork` (run BEFORE either upgrade script), then the two upgrade scripts, then `npx hardhat run scripts/fork/migration_post.ts --network baseFork`	Section 21's migration-safety check — records a real staking position's pending reward and principal before the upgrade, then verifies both are preserved exactly after
`npx hardhat run scripts/fork/role_tests.ts --network baseFork` (after both upgrade scripts)	Section 21's role-scoped tests — trader, manager, and plain-user access, run against the already-upgraded fork
`npx hardhat run scripts/fork/fee_tests.ts --network baseFork` (after both upgrade scripts)	Section 22's manager fee-minting and fee-governance tests, including the real `commitFeeIncrease()` finding
`npx hardhat run scripts/fork/guard_hardening_tests.ts --network baseFork` (after both upgrade scripts)	Section 24's five M-06/FNA-16/FNA-58 targeted tests for the `UniswapV3AssetGuard` fix
`npx hardhat run scripts/fork/m06_isolation_test.ts --network baseFork` (after both upgrade scripts)	Section 24's Test 57 — the follow-up that confirms M-06's own check is independently reachable, not shadowed behind the broader NAV-completeness gate
`npx hardhat run scripts/fork/aave_v4_spoke_integration_test.ts --network baseFork` (after the core-contracts upgrade script)	Section 24's addendum, Tests 58-62 — deploys the dormant AaveV4Spoke guards fresh and tests them against real, live Aave V4 Base infrastructure. Checks for and refuses to run against the pre-upgrade pool (see the addendum's own correction).
`npx hardhat run scripts/fork/aave_v4_tokenization_integration_test.ts --network baseFork` (after the core-contracts upgrade script)	Section 24's addendum, Tests 68-72 — deploys the dormant AaveV4Tokenization guards fresh and tests them against the real, live TOKENIZATION_SPOKE. Same precondition check.
`npx hardhat run scripts/fork/morpho_vault_v2_integration_test.ts --network baseFork` (after the core-contracts upgrade script)	Section 24's addendum, Tests 63-67 — deploys the dormant MorphoVaultV2 guards fresh, creates a real vault through Morpho's live Base factory, and tests against it. Same precondition check.
`npx hardhat test test/MorphoBlueManager.test.ts`	Standard-network unit suite cited for M-11 self-dealing coverage
`npx hardhat test test/UniswapV3NonfungiblePositionGuard.test.ts`	Standard-network unit suite cited for M-14, additional to this report's own real fork tests
`npx hardhat test test/AaveV4*.test.ts test/MorphoVaultV2*.test.ts`	Standard-network unit suites cited for Aave V4 / Morpho Vault V2 (Section 21) — 307 of 307 passing
`npx hardhat test`	Full pre-existing regression suite — 1,366 passing, 2 known pre-existing timing flakes

`SEND=1` only ever broadcasts to whatever is listening on `127.0.0.1`; both upgrade scripts independently refuse to broadcast to real mainnet under `SEND=1` without a second, explicit override that was never set during this work.

---

fUSD Upgrade Verification Report · Base mainnet (chain 8453) · All figures in this report were read directly from either live Base mainnet or a local fork of it; none are estimated.